Lecture 8: BFT Consensus and Blockchain

Christoffer Fink

September 25, 2025



This lecture

- Central topic: (Fault-tolerant) Consensus
- Fundamental limitations of fault-tolerance
- Models and assumptions
  - Failure models
  - Network models
  - What for?
- Examples of concrete protocols and their properties
- Blockchain.
  - Bitcoin
  - A little bit about Ethereum and smart contracts
  - Scalability
  - Sharding

Consensus

- Abstractly: Agreeing on a value
- A “value” can be anything.
- Can be used to implement strong consistency (linearizability) (lecture 6)

Equivalent problems

Consensus

These are all fundamentally the **same problem**.

- (Byzantine) consensus (agree on a specific value)
- (Byzantine) agreement (agree on **some** value)
- Atomic broadcast (all-or-nothing deliver and total order)
- State Machine Replication
- Log replication

Requirements

Consensus

We have two basic requirements

- Liveness:
  - Every correct node eventually decides a value.
  - (Loosely: Something good eventually happens.)
  - Requires that at least one correct node is included in consensus.

Requirements

Consensus

We have two basic requirements

- Liveness:
  - Every correct node eventually decides a value.
  - (Loosely: Something good eventually happens.)
  - Requires that at least one correct node is included in consensus.
- Safety:
  - Every correct node decides the same value.
  - $\forall p, q \in \text{Correct} : (p \text{ decides } v) \land (q \text{ decides } v') \implies v = v'$
  - (Loosely: Nothing bad ever happens.)
  - Requires that enough correct nodes are included so we can't separately reach consensus twice in disjoint groups.

Failure models

- Crash (or stop) fault

Failure models

- Crash (or stop) fault
- **Nodes either work correctly or not at all.**

Failure models

- Crash (or stop) fault
- **Nodes either work correctly or not at all.**
- Seems optimistic! What about bugs!?

Failure models

- Crash (or stop) fault
- **Nodes either work correctly or not at all.**
- Seems optimistic! What about bugs!?
- What if it doesn't crash, but just misbehaves? Keeps "working" but not correctly.

Failure models

- Crash (or stop) fault
- **Nodes either work correctly or not at all.**
- Seems optimistic! What about bugs!?
- What if it doesn't crash, but just misbehaves? Keeps "working" but not correctly.
- What about attackers? (Intentional, malicious misbehavior?)

Failure models

- Crash (or stop) fault
- **Nodes either work correctly or not at all.**
- Seems optimistic! What about bugs!?
- What if it doesn't crash, but just misbehaves? Keeps "working" but not correctly.
- What about attackers? (Intentional, malicious misbehavior?)

- Byzantine fault

Failure models

- Crash (or stop) fault
- **Nodes either work correctly or not at all.**
- Seems optimistic! What about bugs!?
- What if it doesn't crash, but just misbehaves? Keeps "working" but not correctly.
- What about attackers? (Intentional, malicious misbehavior?)

- Byzantine fault
- Byzantine nodes can:
  - Crash

Failure models

- Crash (or stop) fault
- **Nodes either work correctly or not at all.**
- Seems optimistic! What about bugs!?
- What if it doesn't crash, but just misbehaves? Keeps "working" but not correctly.
- What about attackers? (Intentional, malicious misbehavior?)

- Byzantine fault
- Byzantine nodes can:
  - Crash
  - Delay their response (or choose not to respond at all)

Failure models

- Crash (or stop) fault
- **Nodes either work correctly or not at all.**
- Seems optimistic! What about bugs!?
- What if it doesn't crash, but just misbehaves? Keeps "working" but not correctly.
- What about attackers? (Intentional, malicious misbehavior?)

- Byzantine fault
- Byzantine nodes can:
  - Crash
  - Delay their response (or choose not to respond at all)
  - Send duplicate messages

Failure models

- Crash (or stop) fault
- **Nodes either work correctly or not at all.**
- Seems optimistic! What about bugs!?
- What if it doesn't crash, but just misbehaves? Keeps "working" but not correctly.
- What about attackers? (Intentional, malicious misbehavior?)

- Byzantine fault
- Byzantine nodes can:
  - Crash
  - Delay their response (or choose not to respond at all)
  - Send duplicate messages
  - Send bogus messages (either garbage or cleverly engineered, i.e. lies)

Failure models

- Crash (or stop) fault
- **Nodes either work correctly or not at all.**
- Seems optimistic! What about bugs!?
- What if it doesn't crash, but just misbehaves? Keeps "working" but not correctly.
- What about attackers? (Intentional, malicious misbehavior?)

- Byzantine fault
- Byzantine nodes can:
  - Crash
  - Delay their response (or choose not to respond at all)
  - Send duplicate messages
  - Send bogus messages (either garbage or cleverly engineered, i.e. lies)
- **Can misbehave in arbitrary ways!**

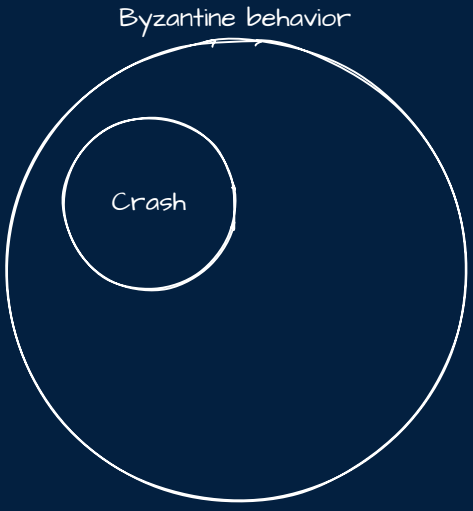
Failure models

- Crash (or stop) fault
- **Nodes either work correctly or not at all.**
- Seems optimistic! What about bugs!?
- What if it doesn't crash, but just misbehaves? Keeps "working" but not correctly.
- What about attackers? (Intentional, malicious misbehavior?)

- Byzantine fault
- Byzantine nodes can:
  - Crash
  - Delay their response (or choose not to respond at all)
  - Send duplicate messages
  - Send bogus messages (either garbage or cleverly engineered, i.e. lies)
- **Can misbehave in arbitrary ways!**
- It's a super set of the crash-fault model.

Failure models

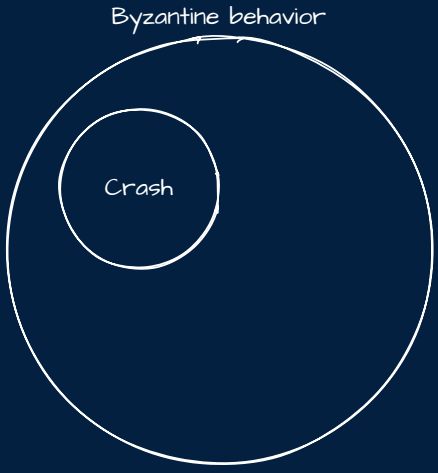
Byzantine faults



A warning about misleading synonyms

Byzantine faults

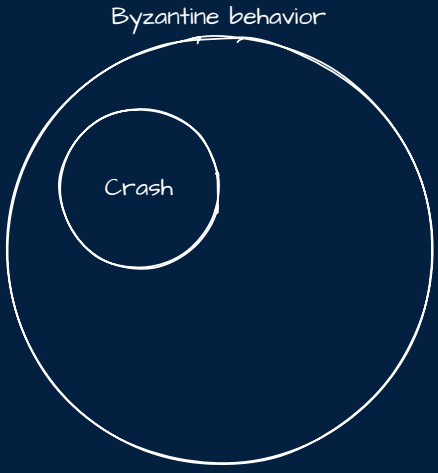
- Some common synonyms:
  - Byzantine, **malicious**, **dishonest**, **traitor**
  - Correct, non-faulty, **honest**, **loyal**



A warning about misleading synonyms

Byzantine faults

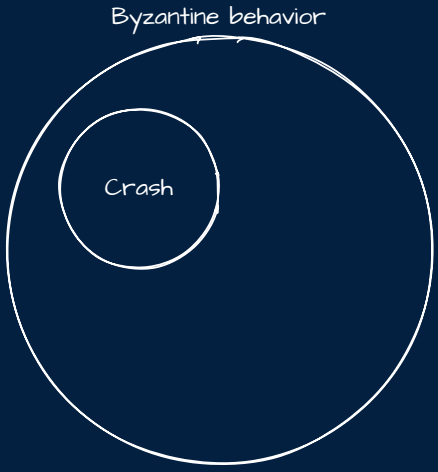
- Some common synonyms:
  - Byzantine, **malicious**, **dishonest**, **traitor**
  - Correct, non-faulty, **honest**, **loyal**
- **These can be misleading!**



A warning about misleading synonyms

Byzantine faults

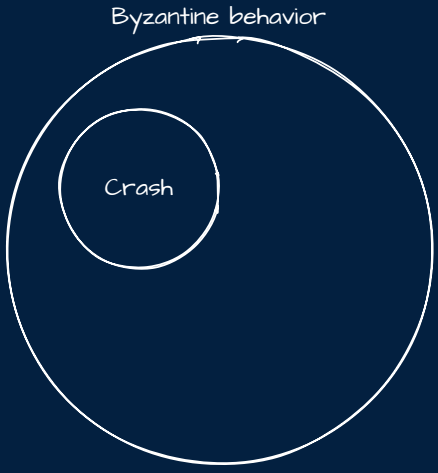
- Some common synonyms:
  - Byzantine, **malicious**, **dishonest**, **traitor**
  - Correct, non-faulty, **honest**, **loyal**
- **These can be misleading!**
- What if an “honest” node crashes?



A warning about misleading synonyms

Byzantine faults

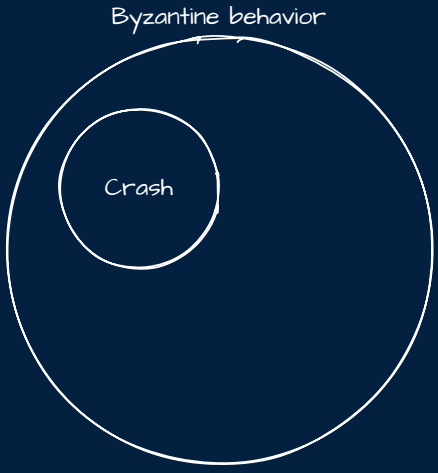
- Some common synonyms:
  - Byzantine, **malicious**, **dishonest**, **traitor**
  - Correct, non-faulty, **honest**, **loyal**
- **These can be misleading!**
- What if an “honest” node crashes?
- If “honest” is a synonym for “non-faulty”, **we can’t have a faulty non-faulty node!**



A warning about misleading synonyms

Byzantine faults

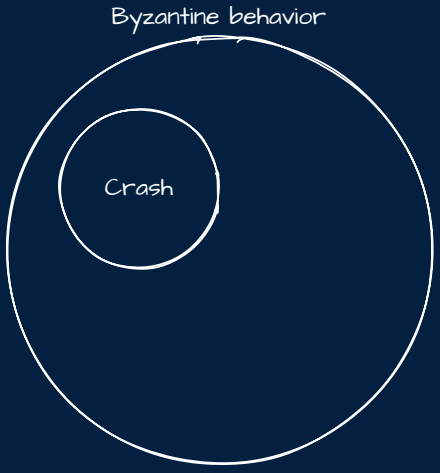
- Some common synonyms:
  - Byzantine, **malicious**, **dishonest**, **traitor**
  - Correct, non-faulty, **honest**, **loyal**
- **These can be misleading!**
- What if an “honest” node crashes?
- If “honest” is a synonym for “non-faulty”, **we can’t have a faulty non-faulty node!**
- If it crashes (no matter how “honest”), it is Byzantine!



A warning about misleading synonyms

Byzantine faults

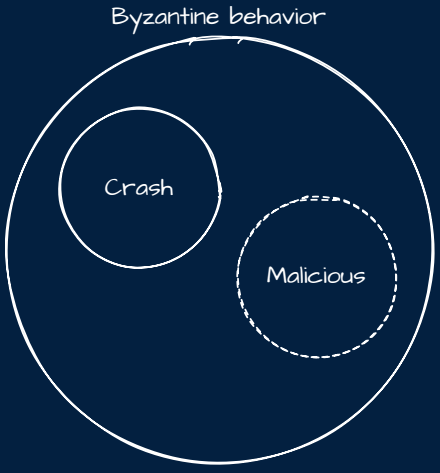
- Some common synonyms:
  - Byzantine, **malicious**, **dishonest**, **traitor**
  - Correct, non-faulty, **honest**, **loyal**
- **These can be misleading!**
- What if an “honest” node crashes?
- If “honest” is a synonym for “non-faulty”, **we can’t have a faulty non-faulty node!**
- If it crashes (no matter how “honest”), it is Byzantine!
- Byzantine $\neq$ malicious



A warning about misleading synonyms

Byzantine faults

- However, the Byzantine model allows us to reason about intentionally malicious behavior.
- Usually it doesn't matter if a node does X on purpose or by accident. What matters is that it can do X.
- But sometimes the distinction is meaningful. (E.g. proving negligible probability.)



Raft

- Goal: Byzantine-fault-tolerant (BFT) consensus
- You already learned about Raft.
- So we'll start there.

Questions?

Raft

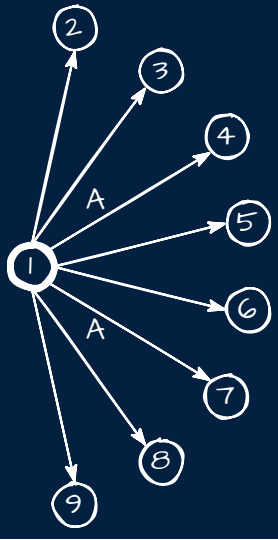
Quick recap

- Servers (nodes) replicate a log of client commands.

Raft

Quick recap

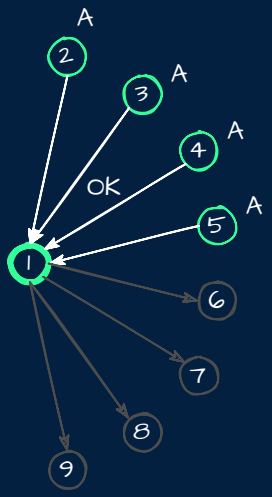
- Servers (nodes) replicate a log of client commands.
- The leader sends new log entries to the followers.



Raft

Quick recap

- Servers (nodes) replicate a log of client commands.
- The leader sends new log entries to the followers.
- When a strict majority has acknowledged an entry/command, the leader commits and tells followers to commit.



Raft

Quick recap

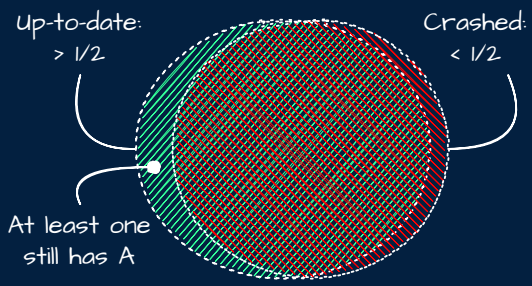
- Servers (nodes) replicate a log of client commands.
- The leader sends new log entries to the followers.
- When a strict majority has acknowledged an entry/command, the leader commits and tells followers to commit.
- Can tolerate that a minority of servers crash.



Raft

Quick recap

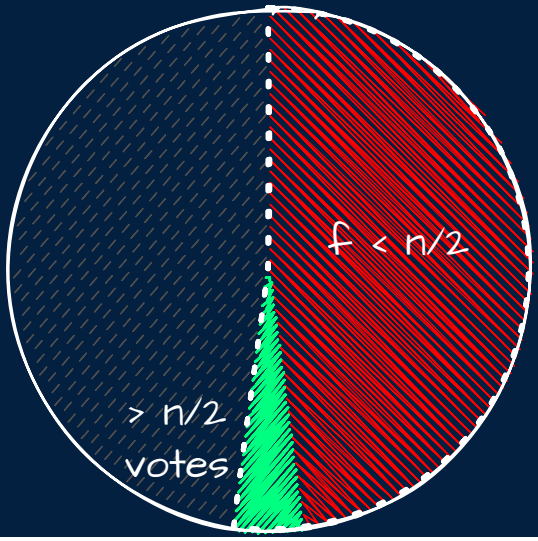
- Servers (nodes) replicate a log of client commands.
- The leader sends new log entries to the followers.
- When a strict majority has acknowledged an entry/command, the leader commits and tells followers to commit.
- Can tolerate that a minority of servers crash.
- Works because at least one that has the data must remain!



Raft

Quick recap

- Servers (nodes) replicate a log of client commands.
- The leader sends new log entries to the followers.
- When a strict majority has acknowledged an entry/command, the leader commits and tells followers to commit.
- Can tolerate that a minority of servers crash.
- Works because at least one that has the data must remain!



Fault tolerance limitations

- Raft can tolerate that $f < n/2$ servers are faulty (crash).
- More precisely, for arbitrary n:
  - Can tolerate $f \leq \lceil \frac{n}{2} \rceil - 1$ crashes.
  - Need a $\geq \lfloor \frac{n}{2} \rfloor + 1$ majority for consensus.
- Assuming f faulty nodes, we need $n \geq 2f + 1$ nodes in total.
- For now, assume $n = 2f + 1$.
- Most efficient.
- We can skip floors/ceilings: tolerate $f < n/2$, need $> n/2$ majority.

Raft

Quick recap

- But Raft is **NOT** Byzantine-fault tolerant.

Raft

Quick recap

- But Raft is **NOT** Byzantine-fault tolerant.
- Completely assumes everyone is “honest”.

Raft

Quick recap

- But Raft is **NOT** Byzantine-fault tolerant.
- Completely assumes everyone is “honest”.
- More accurately: *follows the protocol **correctly** or not at all (crashes)*.

BFT Consensus with $f < n/2$

Scenario 1

- Maybe if we tweak Raft a little, we can tolerate $< 1/2$ Byzantine nodes?
- Assuming leader makes proposals and followers vote.
- One more secret assumption that we will talk about later.

BFT Consensus with $f < n/2$

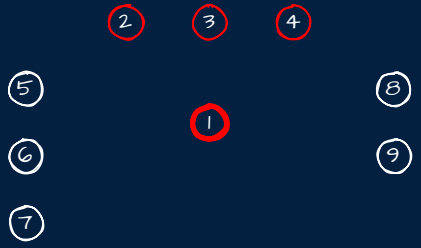
Scenario 1

- Suppose all the Byzantine nodes collude.
  Still $< 1/2$.

BFT Consensus with $f < n/2$

Scenario 1

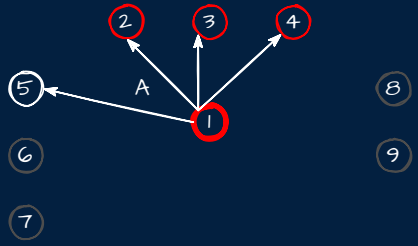
- Suppose all the Byzantine nodes collude. Still $< 1/2$.
- But what if the leader is Byzantine?



BFT Consensus with $f < n/2$

Scenario 1

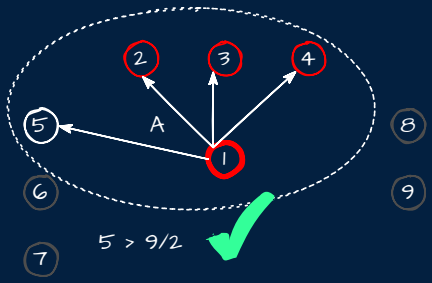
- Suppose all the Byzantine nodes collude. Still $< 1/2$.
- But what if the leader is Byzantine?
- It could propose *A* to a correct node.



BFT Consensus with $f < n/2$

Scenario 1

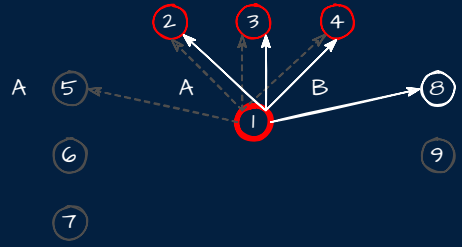
- Suppose all the Byzantine nodes collude. Still $< 1/2$.
- But what if the leader is Byzantine?
- It could propose *A* to a correct node.
- Byzantine + one correct achieves the $> 1/2$ majority.



BFT Consensus with $f < n/2$

Scenario 1

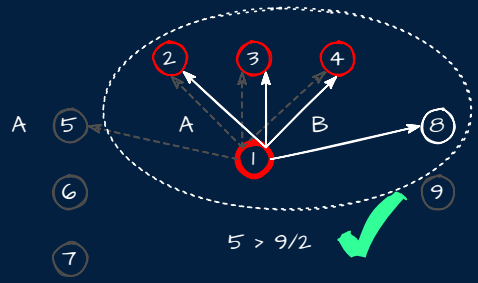
- Suppose all the Byzantine nodes collude. Still $< 1/2$.
- But what if the leader is Byzantine?
- It could propose *A* to a correct node.
- Byzantine + one correct achieves the $> 1/2$ majority.
- Also proposes *B* to another correct node.



BFT Consensus with $f < n/2$

Scenario 1

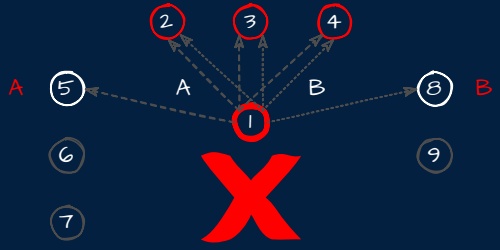
- Suppose all the Byzantine nodes collude. Still $< 1/2$.
- But what if the leader is Byzantine?
- It could propose *A* to a correct node.
- Byzantine + one correct achieves the $> 1/2$ majority.
- Also proposes *B* to another correct node.
- Again a $> 1/2$ majority is achieved.



BFT Consensus with $f < n/2$

Scenario 1

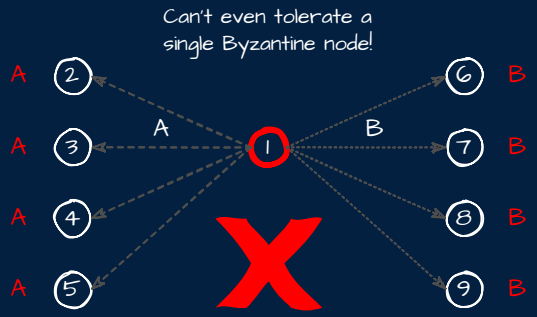
- Suppose all the Byzantine nodes collude. Still $< 1/2$.
- But what if the leader is Byzantine?
- It could propose *A* to a correct node.
- Byzantine + one correct achieves the $> 1/2$ majority.
- Also proposes *B* to another correct node.
- Again a $> 1/2$ majority is achieved.
- Safety violated: *Two **correct** nodes have decided different values!*



BFT Consensus with $f < n/2$

Scenario 1

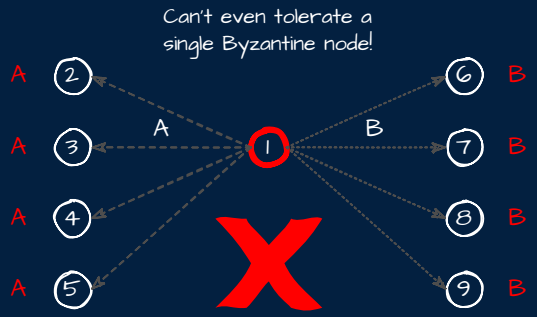
- Didn't even need all the correct nodes to achieve two majorities.



BFT Consensus with $f < n/2$

Scenario 1

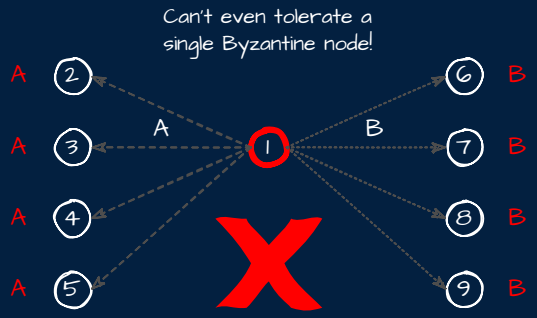
- Didn't even need all the correct nodes to achieve two majorities.
- A **single** Byzantine leader is enough!



BFT Consensus with $f < n/2$

Scenario 1

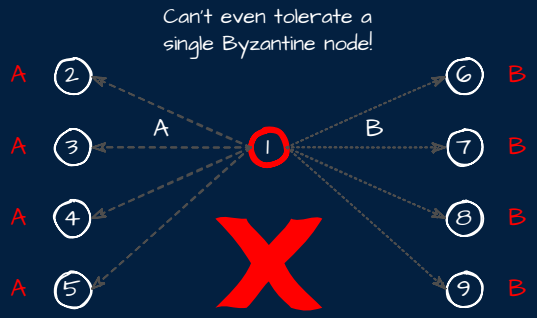
- Didn't even need all the correct nodes to achieve two majorities.
- A **single** Byzantine leader is enough!
- Must be leader.



BFT Consensus with $f < n/2$

Scenario 1

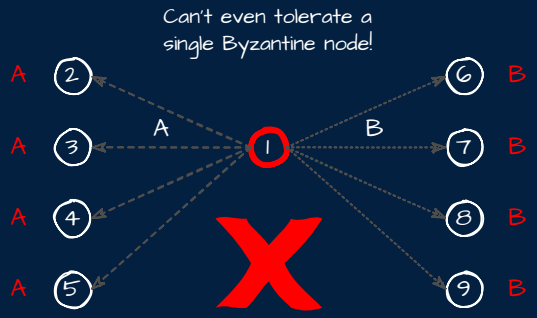
- Didn't even need all the correct nodes to achieve two majorities.
- A **single** Byzantine leader is enough!
- Must be leader.
- Easy fix? Broadcast votes so other correct nodes see the conflicting proposal/vote?



BFT Consensus with $f < n/2$

Scenario 1

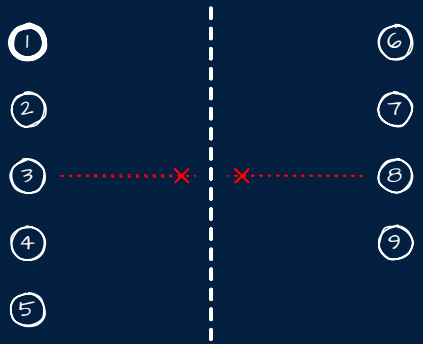
- Didn't even need all the correct nodes to achieve two majorities.
- A **single** Byzantine leader is enough!
- Must be leader.
- Easy fix? Broadcast votes so other correct nodes see the conflicting proposal/vote?
- Not quite that simple.



BFT Consensus with $f < n/2$

Scenario 2

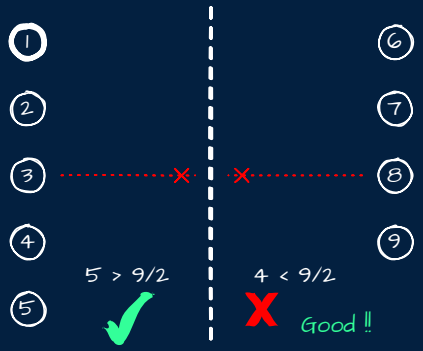
- Suppose there is a network partition, and the nodes are divided into two (almost) even halves.



BFT Consensus with $f < n/2$

Scenario 2

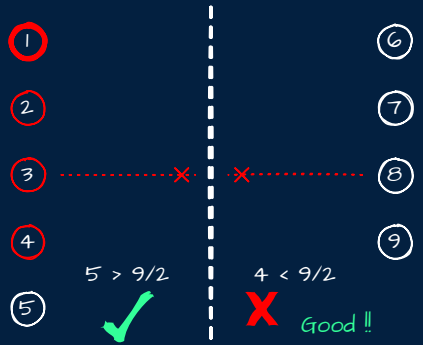
- Suppose there is a network partition, and the nodes are divided into two (almost) even halves.
- Only one half should be able to achieve a majority.



BFT Consensus with $f < n/2$

Scenario 2

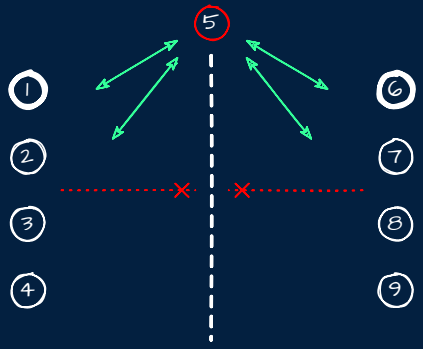
- Suppose there is a network partition, and the nodes are divided into two (almost) even halves.
- Only one half should be able to achieve a majority.
- This is also fine.



BFT Consensus with $f < n/2$

Scenario 2

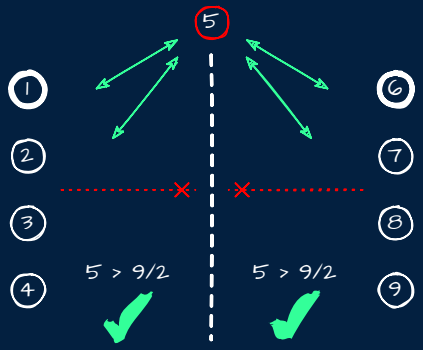
- Suppose a Byzantine node able to communicate with all other nodes!



BFT Consensus with $f < n/2$

Scenario 2

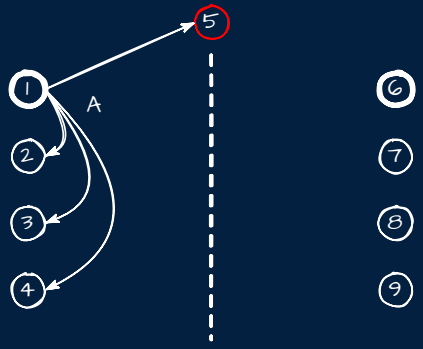
- Suppose a Byzantine node able to communicate with all other nodes!
- It can participate twice!



BFT Consensus with $f < n/2$

Scenario 2

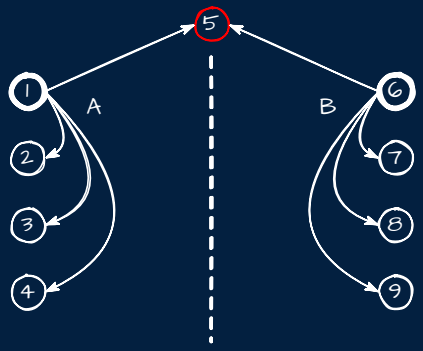
- Suppose a Byzantine node able to communicate with all other nodes!
- It can participate twice!
- Both partitions can have $1/2 + 1$ nodes/votes!



BFT Consensus with $f < n/2$

Scenario 2

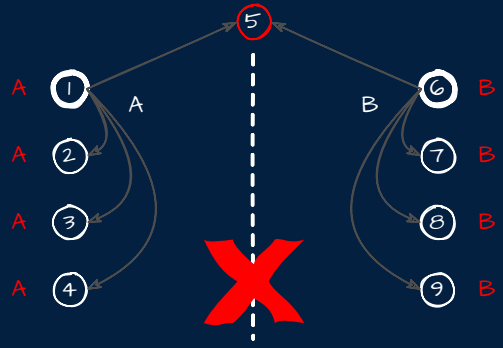
- Suppose a Byzantine node able to communicate with all other nodes!
- It can participate twice!
- Both partitions can have $1/2 + 1$ nodes/votes!
- Safety again violated with a single **non-leader** Byzantine node!



BFT Consensus with $f < n/2$

Scenario 2

- Suppose a Byzantine node able to communicate with all other nodes!
- It can participate twice!
- Both partitions can have $1/2 + 1$ nodes/votes!
- Safety again violated with a single **non-leader** Byzantine node!



BFT Consensus with $f < n/2$

A different threshold?

- The $> 1/2$ majority threshold didn't work.

BFT Consensus with $f < n/2$

A different threshold?

- The $> 1/2$ majority threshold didn't work.
- What if we require unanimous agreement, i.e. all n nodes agree?

BFT Consensus with $f < n/2$

A different threshold?

- The $> 1/2$ majority threshold didn't work.
- What if we require unanimous agreement, i.e. all n nodes agree?
- Well, then the Byzantine nodes can consistently block progress by refusing to respond.

BFT Consensus with $f < n/2$

A different threshold?

- The $> 1/2$ majority threshold didn't work.
- What if we require unanimous agreement, i.e. all n nodes agree?
- Well, then the Byzantine nodes can consistently block progress by refusing to respond.
- We get safety but lose liveness!

BFT Consensus with $f < n/2$

A different threshold?

- The $> 1/2$ majority threshold didn't work.
- What if we require unanimous agreement, i.e. all n nodes agree?
- Well, then the Byzantine nodes can consistently block progress by refusing to respond.
- We get safety but lose liveness!
- We need a threshold that can be achieved with $n - f$ correct nodes.

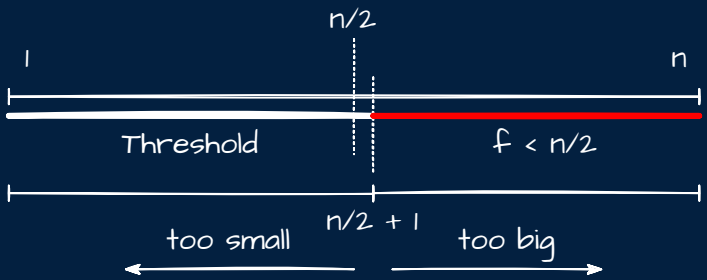
BFT Consensus with $f < n/2$

A different threshold?

- The $> 1/2$ majority threshold didn't work.
- What if we require unanimous agreement, i.e. all n nodes agree?
- Well, then the Byzantine nodes can consistently block progress by refusing to respond.
- We get safety but lose liveness!
- We need a threshold that can be achieved with $n - f$ correct nodes.
- In fact, nothing above $n/2 + 1$ works!

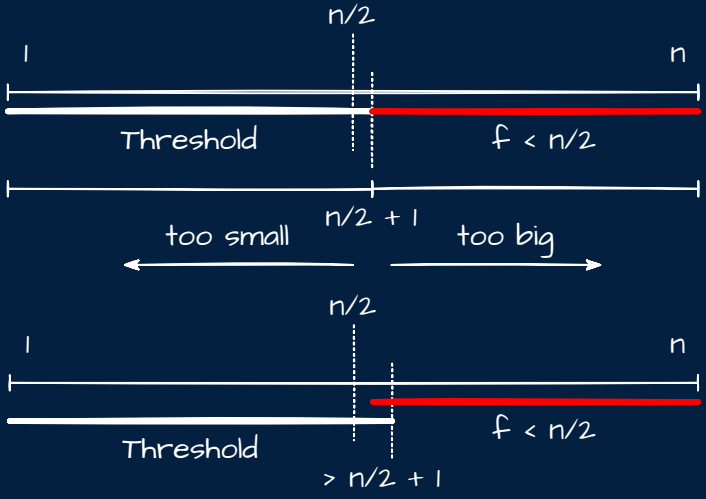
BFT Consensus with $f < n/3$

Three failure cases



BFT Consensus with $f < n/3$

Three failure cases



BFT Consensus with $f < n/2$

Impossible?

- There is no possible threshold that could work.

BFT Consensus with $f < n/2$

Impossible?

- There is no possible threshold that could work.
- So we **cannot tolerate a $< 1/2$ fraction of Byzantine nodes!**

BFT Consensus with $f < n/2$

Impossible?

- There is no possible threshold that could work.
- So we **cannot tolerate a $< 1/2$ fraction of Byzantine nodes!**
- (Well, unless we make some different assumptions.)

BFT Consensus with $f < n/2$

Impossible?

- There is no possible threshold that could work.
- So we **cannot tolerate a $< 1/2$ fraction of Byzantine nodes!**
- (Well, unless we make some different assumptions.)
- What fraction could we tolerate?

BFT Consensus

Three failure cases

- The scenarios rely on correct nodes being unable to communicate with each other due to network problems.

BFT Consensus

Three failure cases

- The scenarios rely on correct nodes being unable to communicate with each other due to network problems.
- We can’t distinguish a node that has crashed from one that is slow to respond.

BFT Consensus

Three failure cases

- The scenarios rely on correct nodes being unable to communicate with each other due to network problems.
- We can’t distinguish a node that has crashed from one that is slow to respond.
- So we have three cases to consider:
  - Node responds

BFT Consensus

Three failure cases

- The scenarios rely on correct nodes being unable to communicate with each other due to network problems.
- We can’t distinguish a node that has crashed from one that is slow to respond.
- So we have three cases to consider:
  - Node responds
  - Node does not respond (yet?)
    - Could have crashed (Byzantine)
    - Could be correct but slow (network problems)

BFT Consensus

Three failure cases

- The scenarios rely on correct nodes being unable to communicate with each other due to network problems.
- We can't distinguish a node that has crashed from one that is slow to respond.
- So we have three cases to consider:
  - Node responds
  - Node does not respond (yet?)
    - Could have crashed (Byzantine)
    - Could be correct but slow (network problems)
- So if some nodes fail to respond, we don't know that they have crashed, and we can't assume the ones that do respond are correct.

BFT Consensus

Three failure cases

- The scenarios rely on correct nodes being unable to communicate with each other due to network problems.
- We can't distinguish a node that has crashed from one that is slow to respond.
- So we have three cases to consider:
  - Node responds
  - Node does not respond (yet?)
    - Could have crashed (Byzantine)
    - Could be correct but slow (network problems)
- So if some nodes fail to respond, we don't know that they have crashed, and we can't assume the ones that do respond are correct.
- Three cases suggests $1/3$ might be a good fraction to try next.

BFT Consensus with $f < n/3$

What threshold?

- Suppose we have $f < n/3$ Byzantine nodes.

BFT Consensus with $f < n/3$

What threshold?

- Suppose we have $f < n/3$ Byzantine nodes.
- Now we assume $n = 3f + 1$.

BFT Consensus with $f < n/3$

What threshold?

- Suppose we have $f < n/3$ Byzantine nodes.
- Now we assume $n = 3f + 1$.
- How many votes do we need now?

BFT Consensus with $f < n/3$

What threshold?

- Suppose we have $f < n/3$ Byzantine nodes.
- Now we assume $n = 3f + 1$.
- How many votes do we need now?
- $n/3 + 1$ would give us liveness, but

BFT Consensus with $f < n/3$

What threshold?

- Suppose we have $f < n/3$ Byzantine nodes.
- Now we assume $n = 3f + 1$.
- How many votes do we need now?
- $n/3 + 1$ would give us liveness, but
- pretty obvious we'll get the same safety problems as before.

BFT Consensus with $f < n/3$

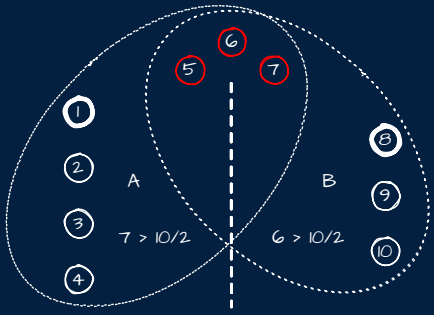
What threshold?

- Suppose we have $f < n/3$ Byzantine nodes.
- Now we assume $n = 3f + 1$.
- How many votes do we need now?
- $n/3 + 1$ would give us liveness, but
- pretty obvious we'll get the same safety problems as before.
- But maybe now the $n/2 + 1$ threshold works?

BFT Consensus with $f < n/3$

$n/2 + 1$ threshold

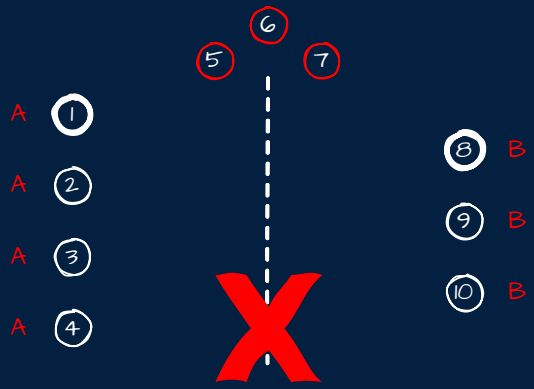
- Partitions still ruin everything!



BFT Consensus with $f < n/3$

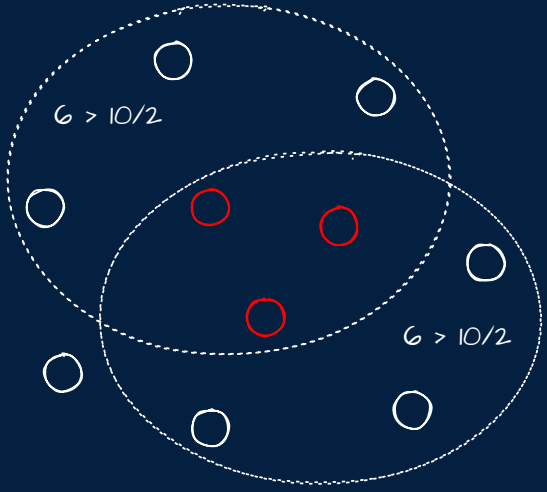
$n/2 + 1$ threshold

- Partitions still ruin everything!



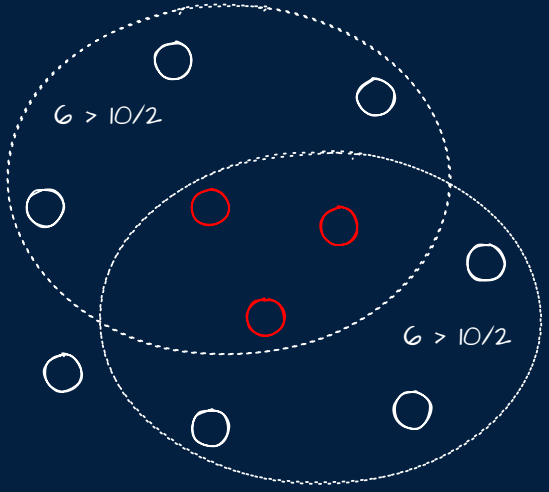
BFT Consensus with $f < n/3$

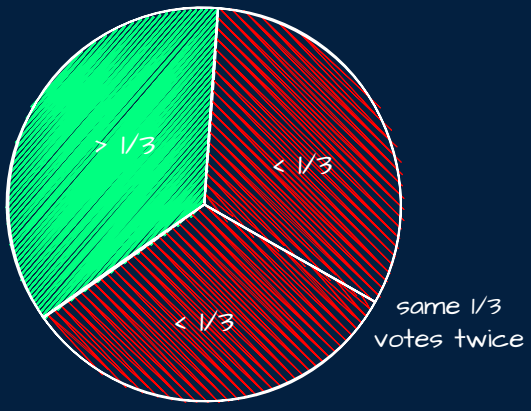
$2n/3 + 1$ threshold



BFT Consensus with $f < n/3$

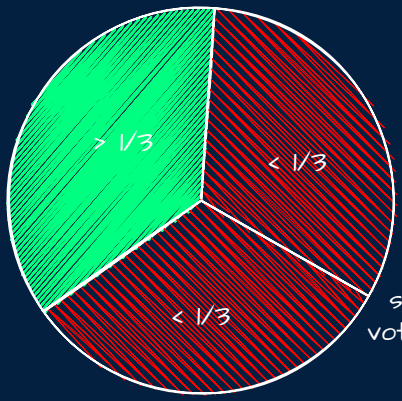
$2n/3 + 1$ threshold



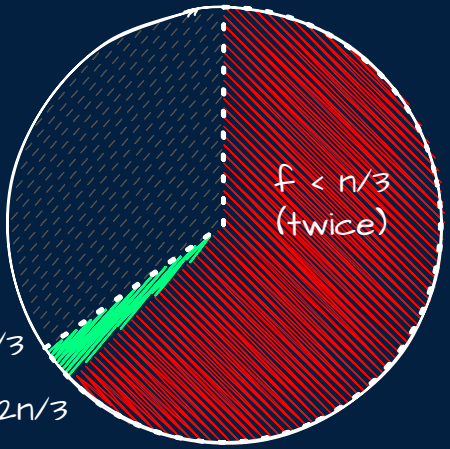


BFT Consensus with $f < n/3$

$2n/3 + 1$ threshold



same 1/3
votes twice



> 2n/3

< 2n/3

BFT Consensus with $f < n/3$

$2n/3 + 1$ threshold

So does a threshold of $2n/3 + 1$ really work?

Let $n = 3f + 1$ and suppose we have (at least) two correct nodes that have committed conflicting values. That means we have achieved two majorities. Correct nodes only vote once, since they never vote for conflicting proposals. But we know Byzantine nodes can vote twice, once for each conflicting proposal. So it's as if there are $f' = 2f$ Byzantine nodes (out of $n' = 4f + 1$ total). That means we can have n' votes in total, counting each correct node once and each Byzantine node twice.

To get one majority, we need this many votes:

$$\begin{aligned}
&\left\lfloor \frac{2n}{3} \right\rfloor + 1 \\
&= \left\lfloor \frac{2(3f+1)}{3} \right\rfloor + 1 \\
&= \left\lfloor \frac{6f}{3} + \frac{2}{3} \right\rfloor + 1 \\
&= \left\lfloor 2f + \frac{2}{3} \right\rfloor + 1 \\
&= 2f + 1
\end{aligned}$$

For two majorities, we need $2(2f + 1) = 4f + 2 > 4f + 1 = n'$. **Impossible!**

Questions?

BFT Consensus with $f < n/2$

Again?

- We can tolerate a fraction $< 1/3$ of Byzantine nodes when we require a $> 2/3$ majority.
- This was based on an implicit assumption: an asynchronous network model.
- This means we can't distinguish crashed nodes from slow nodes.

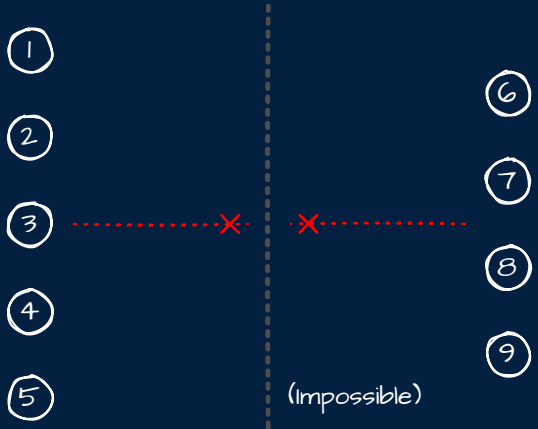
BFT Consensus with $f < n/2$

Synchronous network

- If we could distinguish between crashed and slow nodes, then we could tolerate a $< 1/2$ Byzantine fraction.
- That would require a synchronous network model. (And digital signatures.)
- Concretely: assume there is a **known** upper bound Δ on message propagation.
- In other words: Messages **always** arrive within Δ time.
- If we don't get a response in 2Δ, the node must have crashed!
- Consequence: network partitions are impossible!

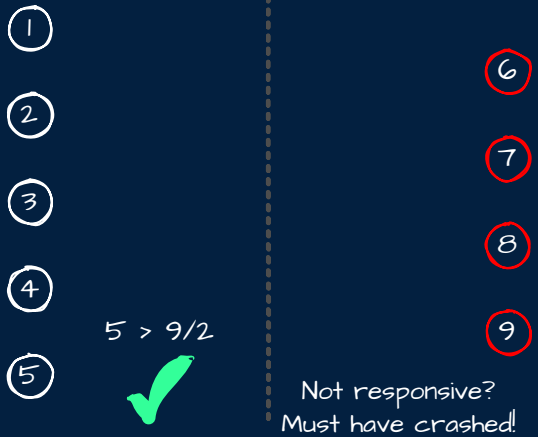
BFT Consensus with $f < n/2$

Synchronous network



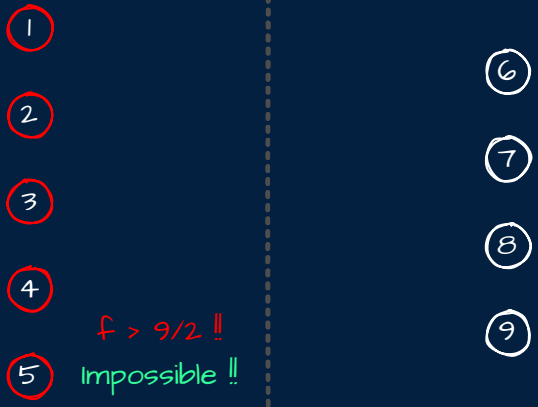
BFT Consensus with $f < n/2$

Synchronous network



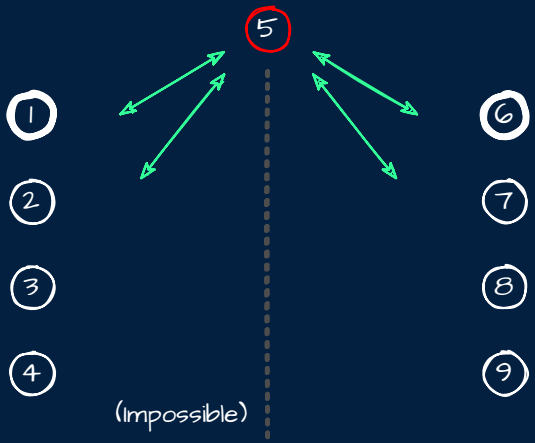
BFT Consensus with $f < n/2$

Synchronous network



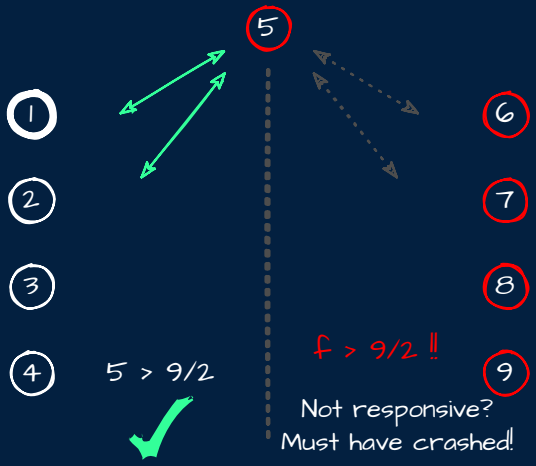
BFT Consensus with $f < n/2$

Synchronous network



BFT Consensus with $f < n/2$

Synchronous network



BFT Consensus with $f < n/2$

Synchronous network

- Under the synchronous assumption, we **can** tolerate a fraction $< 1/2$ of Byzantine nodes when we require a $> 1/2$ majority.

Network models

- With a fully asynchronous model, we run into the famous FLP impossibility result:
  - Liveness cannot be guaranteed with a deterministic protocol in a fully asynchronous network.
  - Can't tolerate a single crash-fault. (Don't even need Byzantine behavior.)
- Most (popular) protocols assume a **partially** (a)synchronous network.
- Middle ground between fully synchronous/asynchronous.
- Don't need to assume synchrony for safety but can rely on synchrony for liveness.
- There exists an **unknown** upper bound Δ. (Or unknown GST.)
- These are all in the $f < n/3$ category.

Summary

BFT Consensus limits

- CFT: can tolerate $< 1/2$ Byzantine fraction (needs a $> 1/2$ majority)
- BFT:
  - Partially (a)synchronous: $< 1/3$ Byzantine ($> 2/3$ majority)
  - Synchronous*: $< 1/2$ Byzantine ($> 1/2$ majority)

* Assuming digital signatures.

Questions?

Concrete protocols

- PBFT
- Tendermint
- HotStuff(-2)
- Desirable properties:
  - Optimistic responsiveness: $O(\delta)$ per decision. ($\delta =$ message delay)
  - Optimal optimistic communication: $O(n)$ per decision.
  - Optimal worst-case communication: $O(n^2)$ per decision.
  - Load balance.

PBFT (Practical Byzantine Fault Tolerance

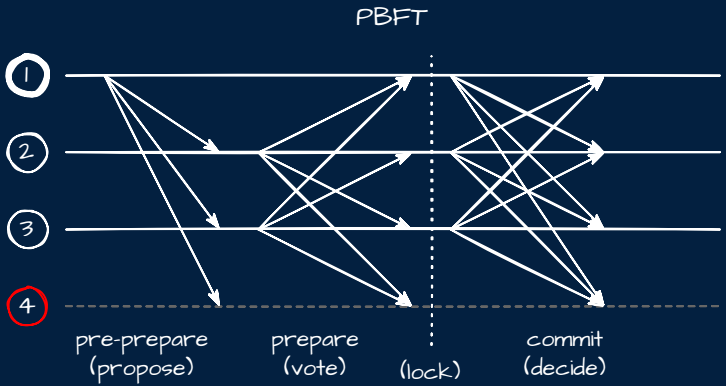
Concrete protocols

- Assumes partial synchrony
  - Tolerates $f < n/3$ Byzantine nodes.
  - Requires $> 2n/3$ majority.
- Similar to Raft:
  - “Views” (just like “terms” in Raft).
  - Each *view* has a stable primary (leader).
  - A view change happens when a primary fails (similar to Raft election, but not the same)
- Differences:
  - Primaries are chosen deterministically. (Round robin. No voting.)
  - The primary sends a proposal.
  - Backups broadcast votes.
  - Three rounds:
    - Pre-Prepare (proposal)
    - Prepare (vote)
    - Commit

PBFT (Practical Byzantine Fault Tolerance)

Concrete protocols

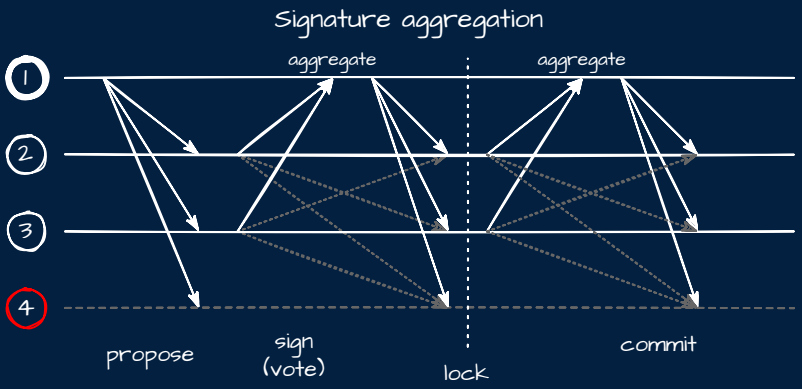
- Optimistic responsiveness: $O(\delta)$ per decision. (δ = message delay)
- $\Theta(n^2)$ messages. (We can do better.)



PBFT with Signature aggregation

Concrete protocols

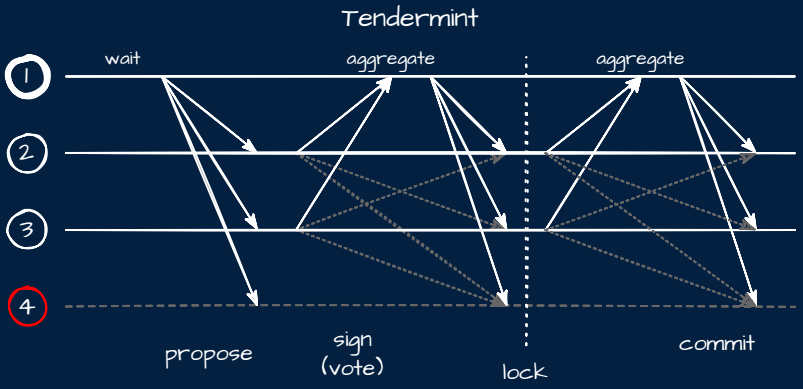
- Optimistic linear communication, i.e. $O(n)$ message complexity.



Tendermint

Concrete protocols

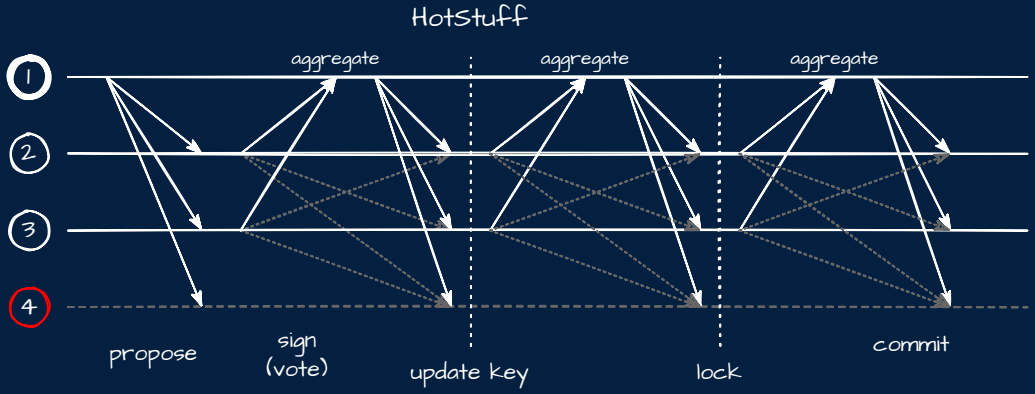
- Two steps forward, one step back.
- **Gain** load balance and linear view change.
- But **lose** optimistic responsiveness: $O(\Delta)$ per decision. (Δ = max message delay)



HotStuff

Concrete protocols

- One step forward, one step back.
- **Regains** the optimistic responsiveness we lost with Tendermint. (Also easy pipelining.)
- But needs an **extra round (4)**.



HotStuff-2

Concrete protocols

- One step forward. (And maybe back?)
- Back to 3 rounds.
- Has all the nice properties, without adding extra rounds.
- However, it turns out HotStuff-2 performance degrades more quickly as the number of Byzantine nodes increases.

Electronic currency

Transactions

Use case: Electronic currency

Validating Transactions

- Basic requirement: Can only spend money that **you** have.
- No forged transactions.

Validating Transactions

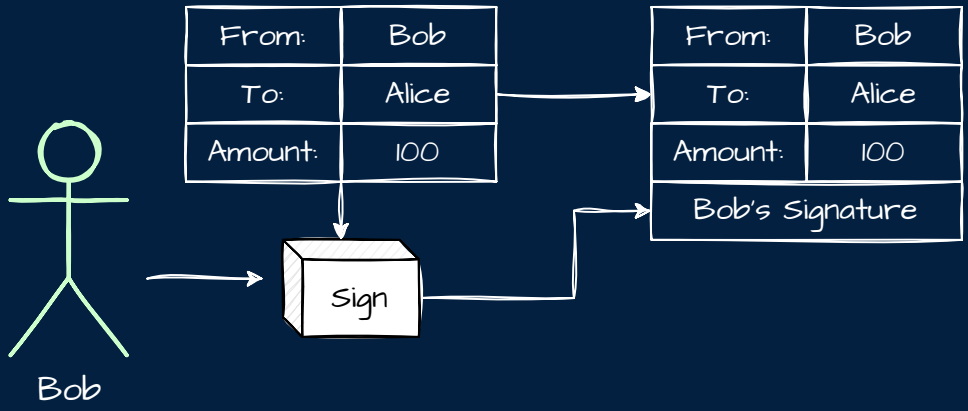


Validating Transactions

- Basic requirement: Can only spend money that **you** have.
- No forged transactions.
- **Solution:** (digital) **signatures.**

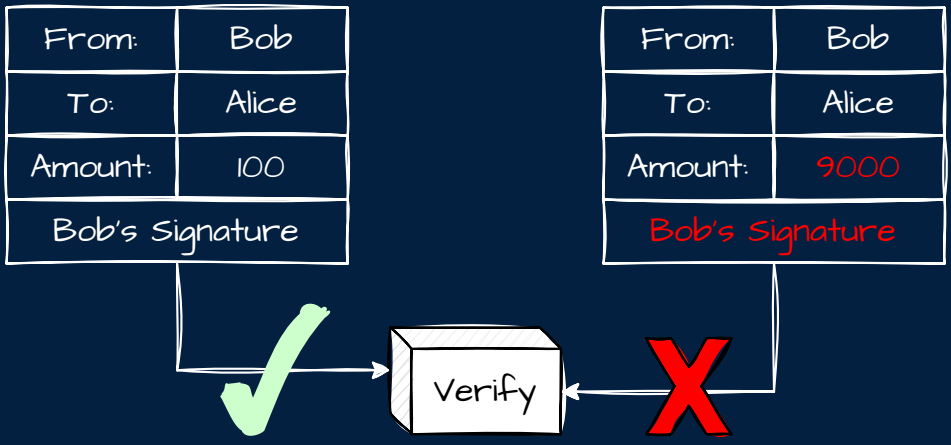
Signatures

Validating Transactions



Signatures

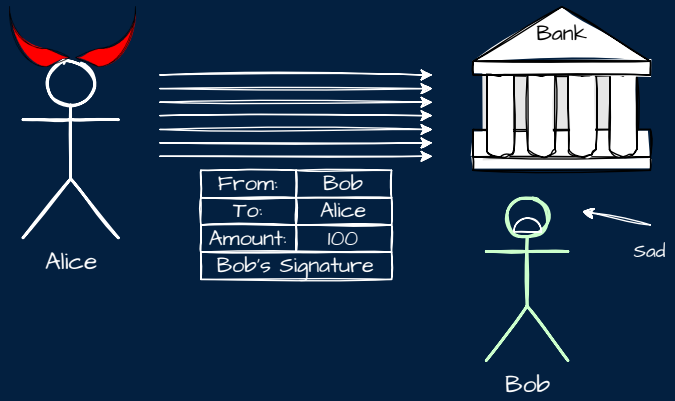
Validating Transactions



Replay Attacks

Validating Transactions

- What about replay attacks?
- Solution: Signatures **+ nonce**.



Validating Transactions

- Basic requirement: Can only spend money that you **have**.

Validating Transactions

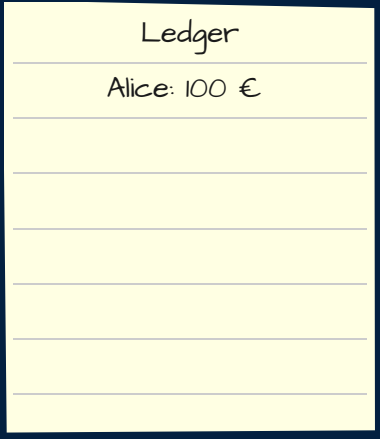
- Basic requirement: Can only spend money that you **have**.
- Special case: No double spending.

Validating Transactions

- Basic requirement: Can only spend money that you **have**.
- Special case: No double spending.
- Solution: Record all transactions on a ledger. (**Consensus!**)

Ledgers

Validating Transactions





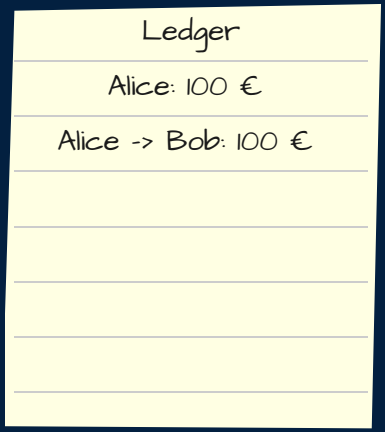
Alice

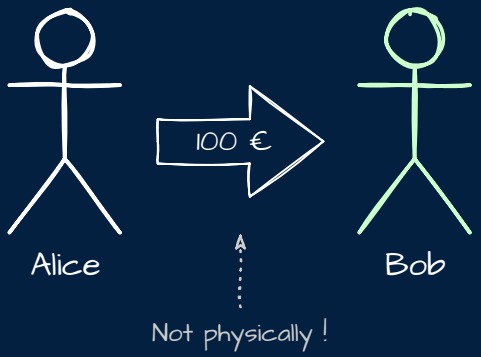


Bob

Ledgers

Validating Transactions





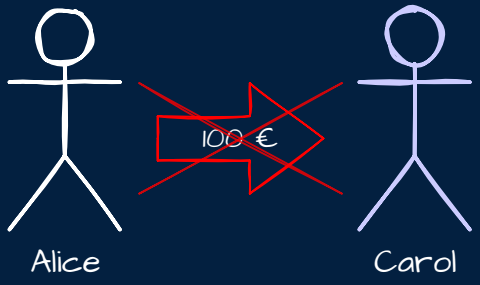
Ledgers

Validating Transactions

Ledger

Alice: 100 €

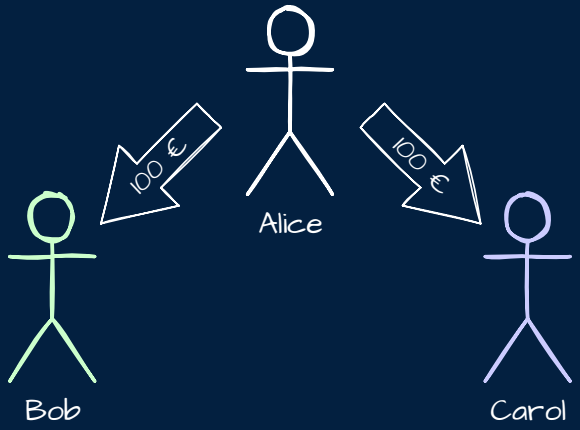
Alice -> Bob: 100 €



Double Spending

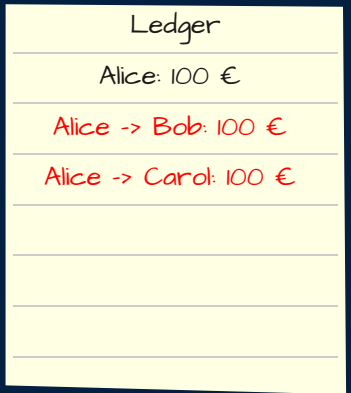
Validating Transactions





Double Spending

Validating Transactions







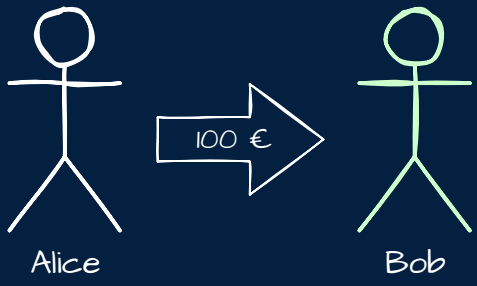


Double Spending

Validating Transactions

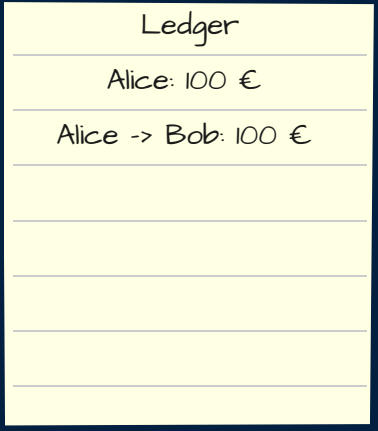
Ledger

Alice: 100 €

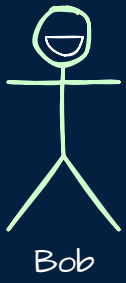


Double Spending

Validating Transactions







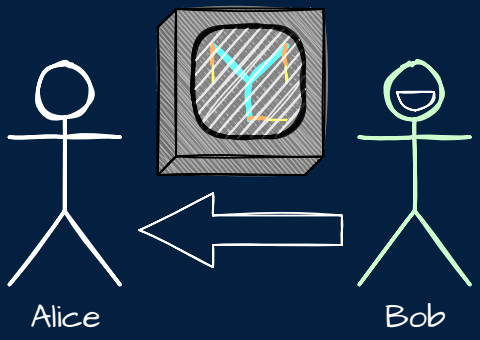
Double Spending

Validating Transactions

Ledger

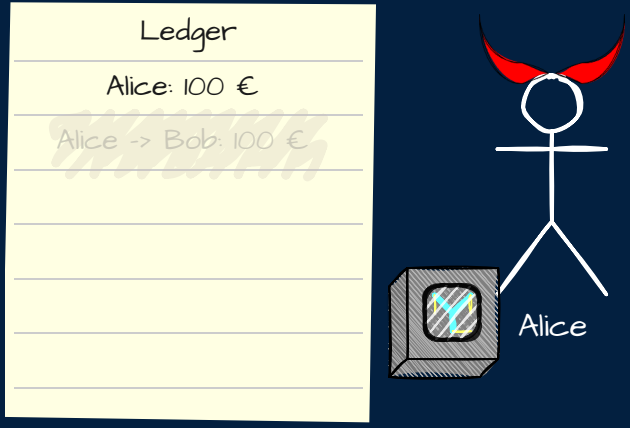
Alice: 100 €

Alice -> Bob: 100 €



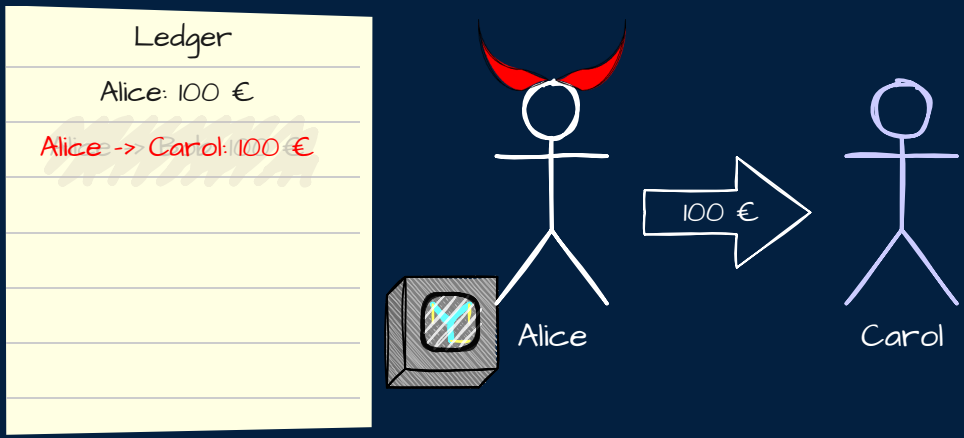
Double Spending

Validating Transactions



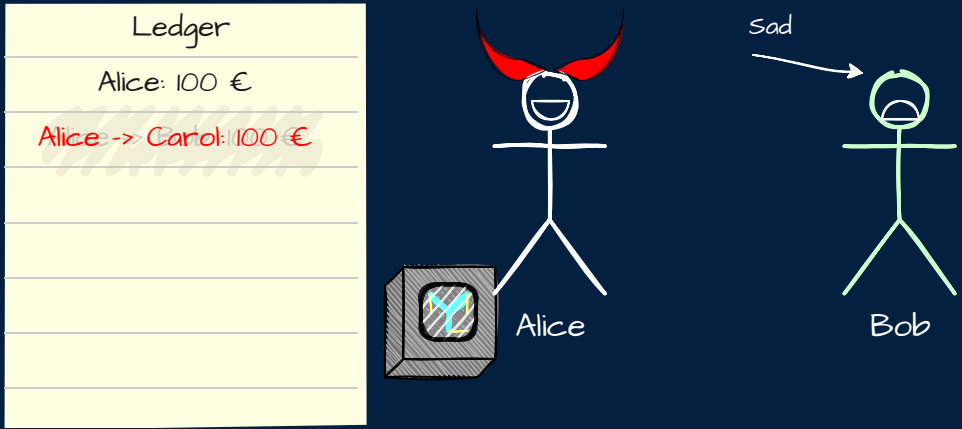
Double Spending

Validating Transactions



Double Spending

Validating Transactions



Validating Transactions

- Basic requirement: Can only spend money that you **have**.
- Special case: No double spending.
- Solution: Record all transactions on a ledger.
  - **Must be immutable (append only).**
  - **Must coordinate multiple updates.**
- If everyone can **agree** on which transactions have happened, validation becomes easy.

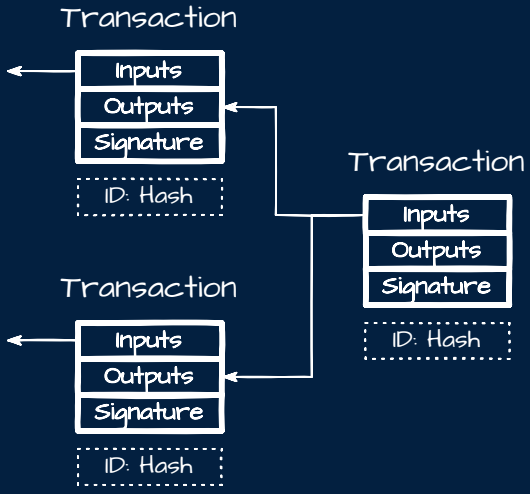
Blockchains

- Blockchains implement an immutable, distributed (and decentralized) ledger.
- First and most famous: Bitcoin.

Transactions

Bitcoin

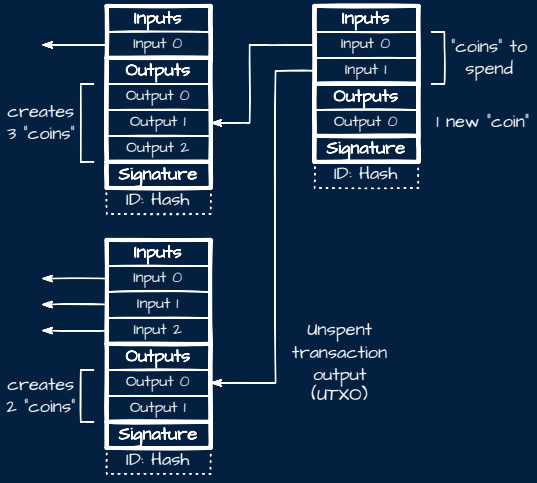
- Transactions reference inputs and outputs. Its hash works as ID.



Transactions

Bitcoin

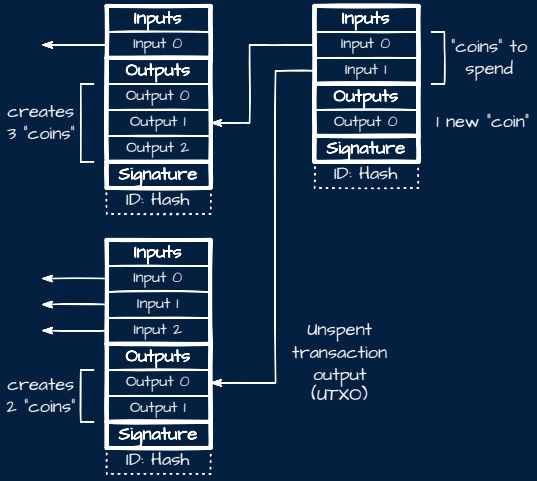
- Transactions reference inputs and outputs. Its hash works as ID.
- An input is an output of a previous transaction (UTXO).



Transactions

Bitcoin

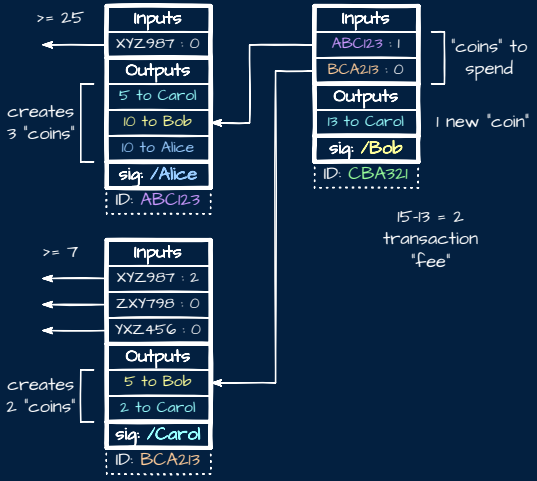
- Transactions reference inputs and outputs. Its hash works as ID.
- An input is an output of a previous transaction (UTXO).
- Transactions spend (use up) UTXOs and create new ones (outputs).



Transactions

Bitcoin

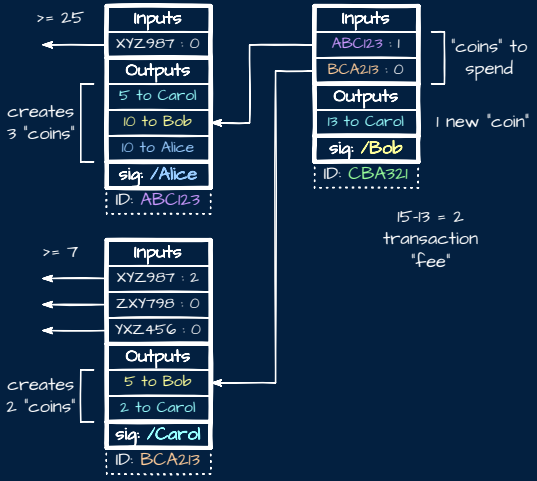
- Transactions reference inputs and outputs. Its hash works as ID.
- An input is an output of a previous transaction (UTXO).
- Transactions spend (use up) UTXOs and create new ones (outputs).
- Outputs specify an amount and a receiver.



Transactions

Bitcoin

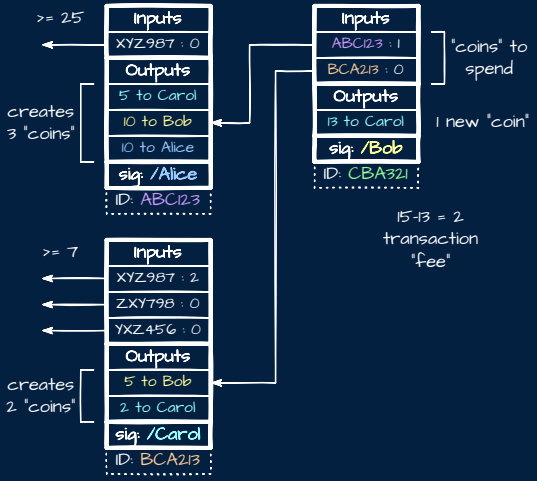
- Transactions reference inputs and outputs. Its hash works as ID.
- An input is an output of a previous transaction (UTXO).
- Transactions spend (use up) UTXOs and create new ones (outputs).
- Outputs specify an amount and a receiver.
- A UTXO is effectively a “coin” (bill, with a unique serial number).



Transactions

Bitcoin

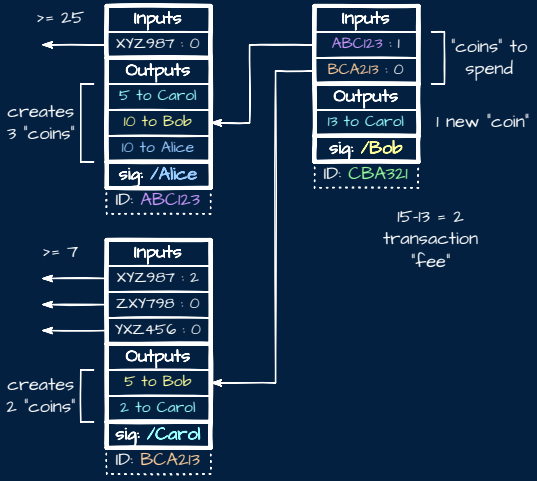
- Transactions reference inputs and outputs. Its hash works as ID.
- An input is an output of a previous transaction (UTXO).
- Transactions spend (use up) UTXOs and create new ones (outputs).
- Outputs specify an amount and a receiver.
- A UTXO is effectively a “coin” (bill, with a unique serial number).
- Spend only once and only the whole coin.



Transactions

Bitcoin

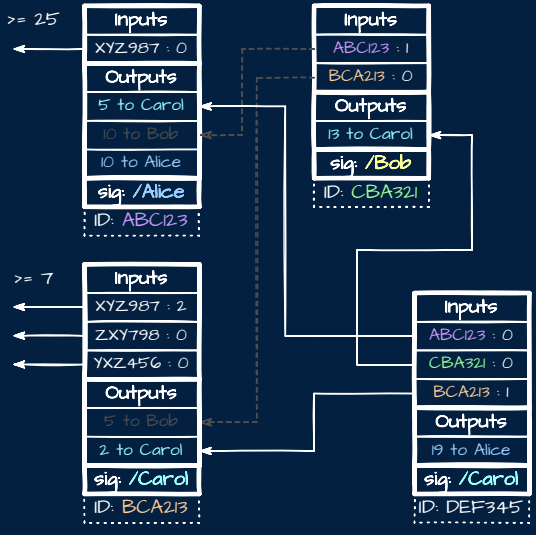
- Transactions reference inputs and outputs. Its hash works as ID.
- An input is an output of a previous transaction (UTXO).
- Transactions spend (use up) UTXOs and create new ones (outputs).
- Outputs specify an amount and a receiver.
- A UTXO is effectively a “coin” (bill, with a unique serial number).
- Spend only once and only the whole coin.
- Change back is an output to self. Excess is paid to the miner.



Transactions

Bitcoin

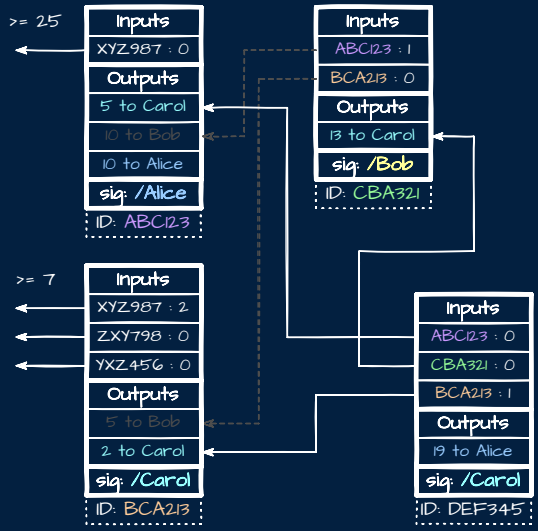
- Transactions reference inputs and outputs. Its hash works as ID.
- An input is an output of a previous transaction (UTXO).
- Transactions spend (use up) UTXOs and create new ones (outputs).
- Outputs specify an amount and a receiver.
- A UTXO is effectively a “coin” (bill, with a unique serial number).
- Spend only once and only the whole coin.
- Change back is an output to self. Excess is paid to the miner.



Transactions

Bitcoin

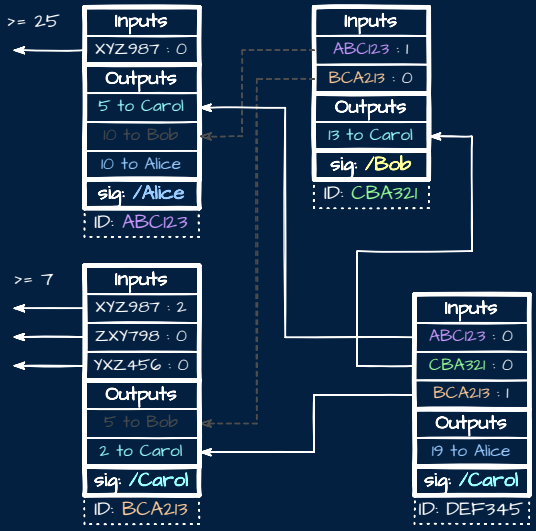
- Coins are effectively split and merged. Destroying coins and creating new ones.



Transactions

Bitcoin

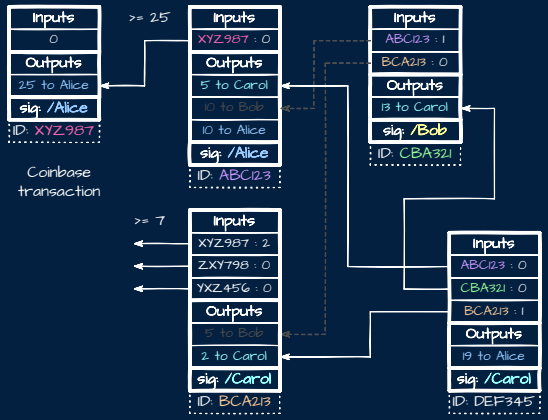
- Coins are effectively split and merged. Destroying coins and creating new ones.
- We can trace the splitting and merging.



Transactions

Bitcoin

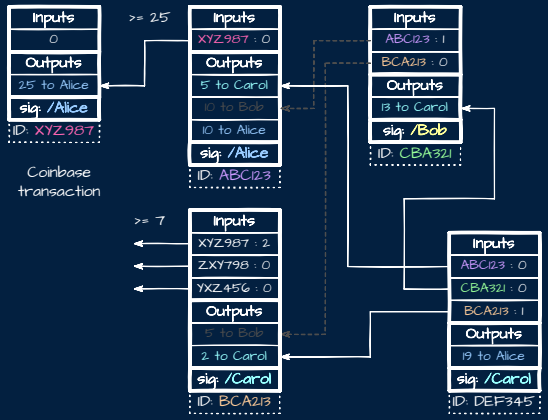
- Coins are effectively split and merged. Destroying coins and creating new ones.
- We can trace the splitting and merging.
- Ultimately starts with a coinbase transaction.



Transactions

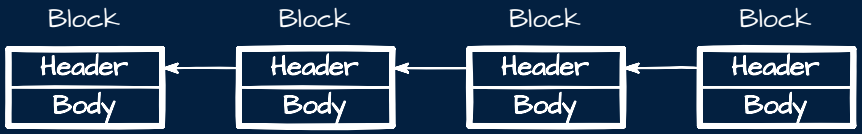
Bitcoin

- Coins are effectively split and merged. Destroying coins and creating new ones.
- We can trace the splitting and merging.
- Ultimately starts with a coinbase transaction.
- Original coins are created by mining.



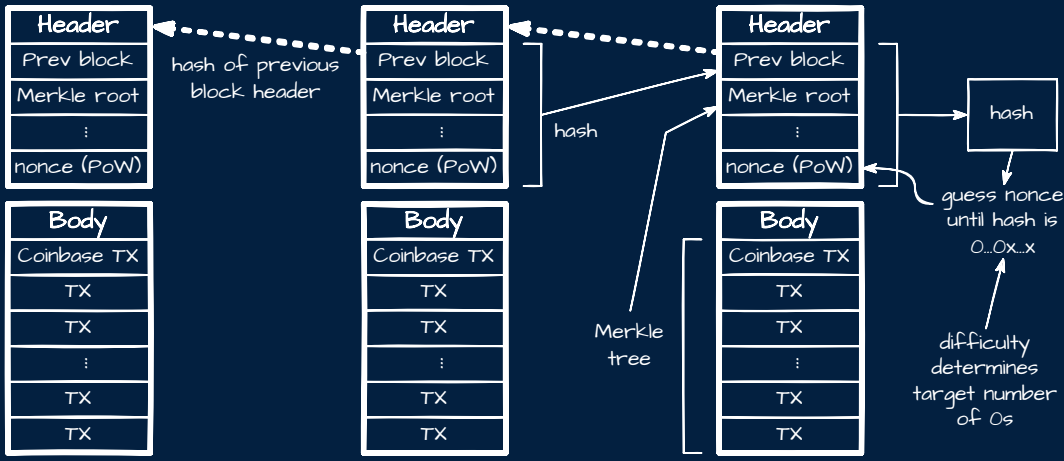
Blockchain

Bitcoin



Blockchain

Bitcoin



Mining

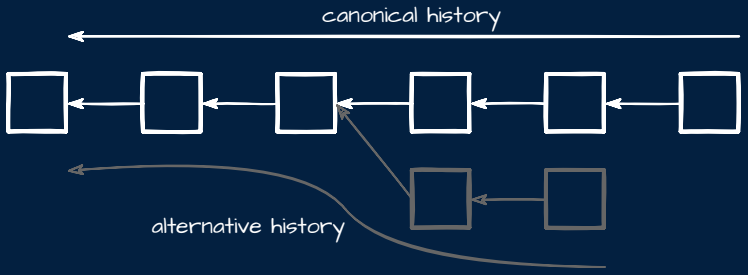
Bitcoin

- Transactions are gossiped through the network.
- At any time, there is a pool of submitted (but not yet processed) transactions.
- Miners pick transactions from this pool and include them in a **block**.
- The block references the previous block in the chain by its hash.
- They also include a special transaction where they give themselves free money. (The block reward.)
- Then they add a magic number to the block.
- This number is chosen such that when the whole block is hashed, the hash starts with n zeros.
- We don't know how to predict which number will work. So you have to guess by brute force.
- By including this number, the miner proves that the necessary effort was spent on producing the block: a **proof-of-work** (PoW).
- Then they gossip this block to the network and other nodes add it to the blockchain.

Consensus

Bitcoin

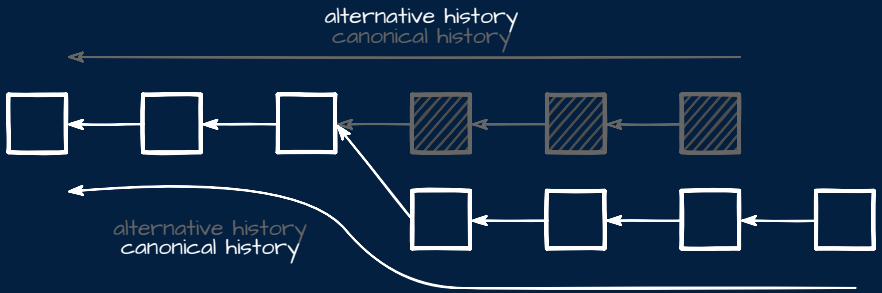
- There can be many forks and therefore many chains.
- Each is a possible history.
- The longest one is the canonical chain; the “true” history everyone agrees on.



Consensus

Bitcoin

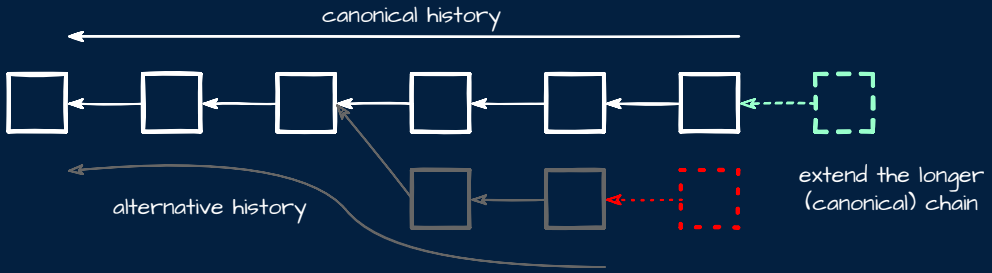
- If we can make another chain longer, it becomes the “true” history.
- The transactions in the tail of the old history didn’t happen!
- History has been rewritten!



Consensus

Bitcoin

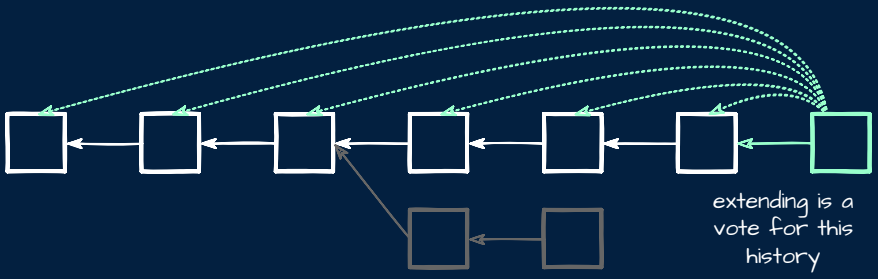
- Consensus protocol: Accept/extend the longest chain.



Consensus

Bitcoin

- The new block is a proposal but also a vote for the whole history it extends.



Consensus

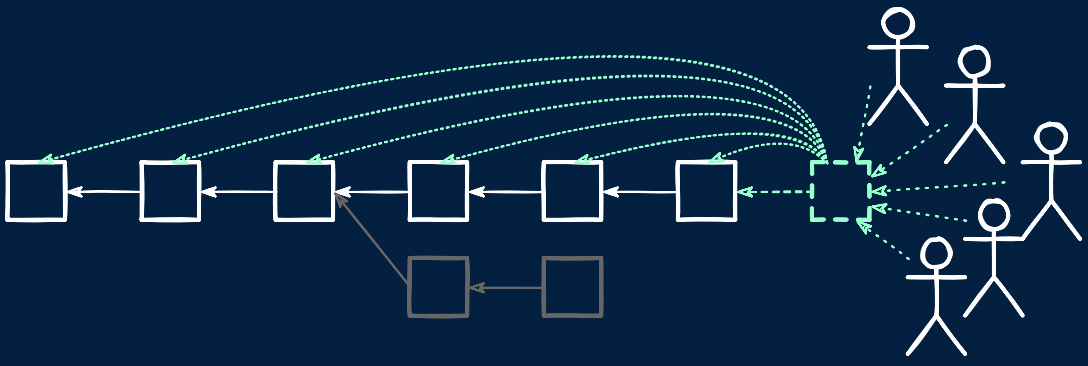
Bitcoin

- No leader.
- There is no explicit voting that ends with accepting/rejecting the proposal (like in PBFT etc.).
- Proposals and votes are combined, and votes are implicit.
- By proposing a block, you implicitly vote for all the previous blocks in the chain it extends.
- Everyone implicitly votes for the proposal by mining new blocks that extend it. (Making new proposals that accept that proposal.)

Consensus

Bitcoin

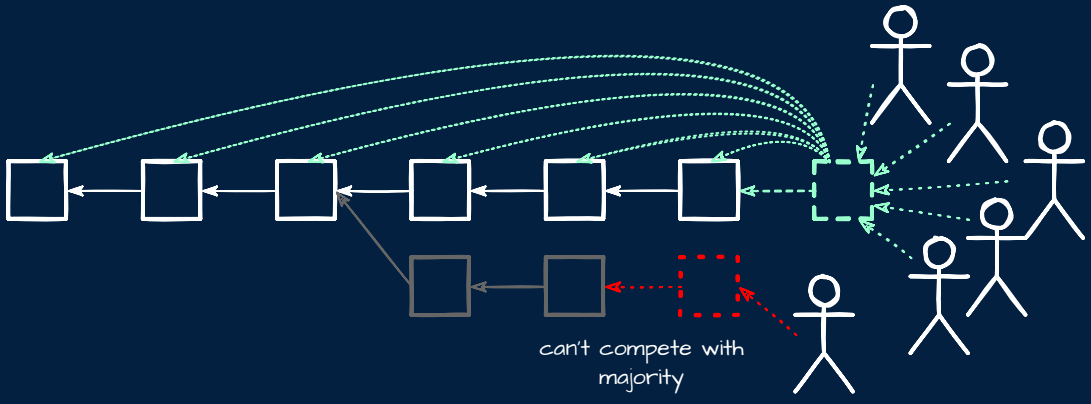
- A single vote can represent the majority! (Everyone is trying to extend the same chain.)



Consensus

Bitcoin

- An attacker has to compete with the “honest” (correct) majority.



Consensus

Bitcoin

- An attacker that controls a majority ($> 50\%$) of the computational resources (hashing power) in the network could
  - out-pace everyone else (mine blocks faster)
  - thereby decide what goes on the longest chain
  - some accounts/transactions can be censored (the attacker just refuses to include them)
  - worse: the attacker can rewrite history by forking off at an earlier point and then extend that chain faster than everyone else is extending the canonical chain.
- Old and “confirmed” transactions suddenly didn’t happen after all!
- A correct majority (of computation) can outperform an attacker.
- Economic incentive for honesty. Only earn money if your block is accepted.
- Can tolerate $< 1/2$ Byzantine.

Consensus

Bitcoin

- Some people refer to Proof-of-Work, Proof-of-Stake, etc. as “consensus protocol” or “consensus mechanism”.
- That is **not accurate!** (Just think about forks and the definition of the safety property!)
- The mechanism to reach **consensus** in Bitcoin is “correct nodes vote for the longest chain”.
- PoW limits how quickly proposals can be created, allowing time for propagation.
- Proposing/voting is **expensive**! Can’t afford to pretend to be several people.

Limitations

Bitcoin

- Fixed block size and fixed block interval means a fixed throughput ($\sim$ 17 TPS theoretical maximum, $\sim$ 7 realistic maximum).
- Everyone replicates the whole state.
- Everyone mines the “same” block. (Compete to add the next block.)
- So everyone is involved in global consensus.
- Adding more nodes does not help! No horizontal scalability.
- Proof-of-work is energy intensive. Resource waste is the purpose!

Ethereum

- No UTXO. Uses the account/balance model.
- Originally used PoW but switched to Proof-of-Stake (PoS).
- Need to “stake” money: put your money on the line.
- Probability of participating depends on amount staked.
- Stake gets destroyed “slashed” if misbehaving.
- Still the same 12 second block interval. So throughput ($\sim$ 30 TPS) did not change!
- But much more energy efficient!

Beyond account balance

Ethereum

- Transactions alter account balance. Basically:
  - `sender.account -= tx.amount`
  - `receiver.account += tx.amount`
- What if we had more variables, not just `account`?
- What if there were other operations, not just add/subtract symmetrically?

Smart contracts

Ethereum

- Then we could have “smart contracts”.
- Most basic version: Transactions that occur automatically when certain conditions are met.
- But we can generalize beyond that to run arbitrary programs.
- Ethereum is a virtual machine that executes code!
- Regular/simple transactions are just a special case.

Limitations of established blockchain systems

Established blockchain systems like Bitcoin and Ethereum have multiple limitations:

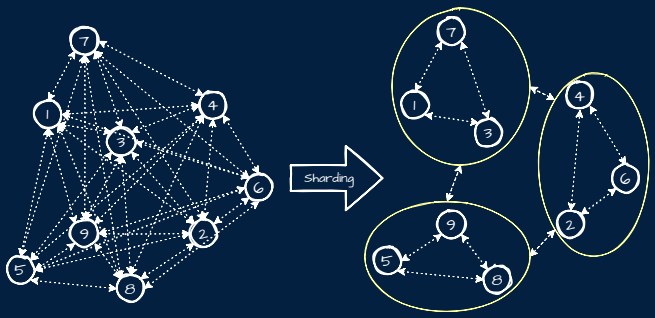
- Large transaction fees (tens of dollars per transaction).
- Low throughput (7-30 transactions per second).
- No horizontal scalability.
- Especially for Bitcoin:
  - Long transaction confirmation delay.
  - Huge power consumption.

Possible scalability solutions

- Layer-one solutions: modify or replace the existing system.
  - Increasing block size.
  - Reduced block interval.
  - **Sharding**.
- Layer-two solutions: implement a new system on top of the existing one.
  - State/Payment channels (e.g. Bitcoin Lightning Network and Raiden).
  - Side chains.
  - Rollups. (+ (Proto)Danksharding)
  - etc.

Sharding

- Divide the network nodes into subsets (“shards”).
- Nodes only validate a subset of transactions.
- Full sharding: Nodes only **store** a subset of transactions.



Sharding challenges

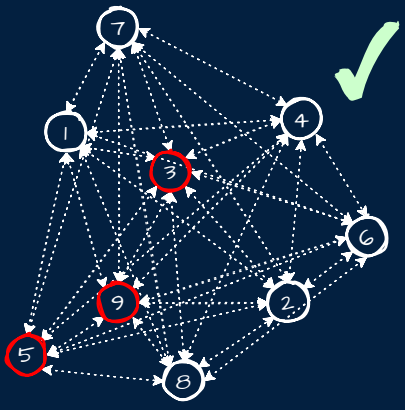
Sharding sounds promising, but also presents some challenges:

- Defining shards, i.e. mapping nodes to shards.
- Sharding introduces a security-performance trade-off.
- Cross-shard validation.

Security-performance trade-off

Sharding challenges

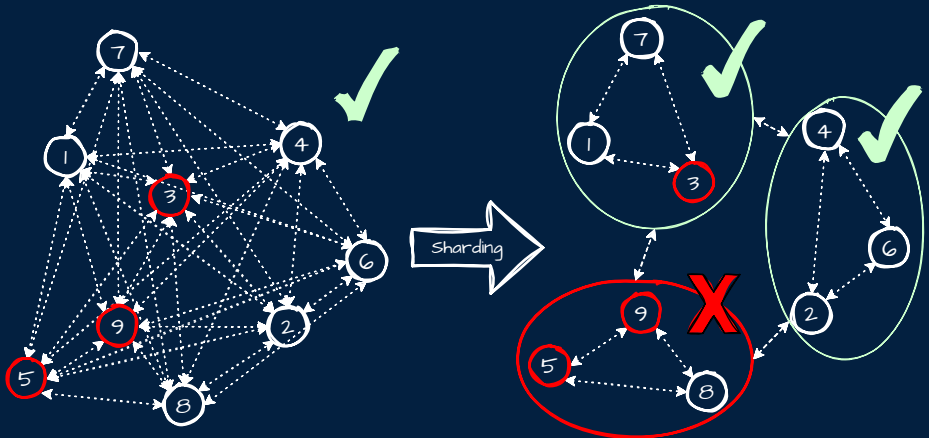
Sharding introduces a security-performance trade-off.



Security-performance trade-off

Sharding challenges

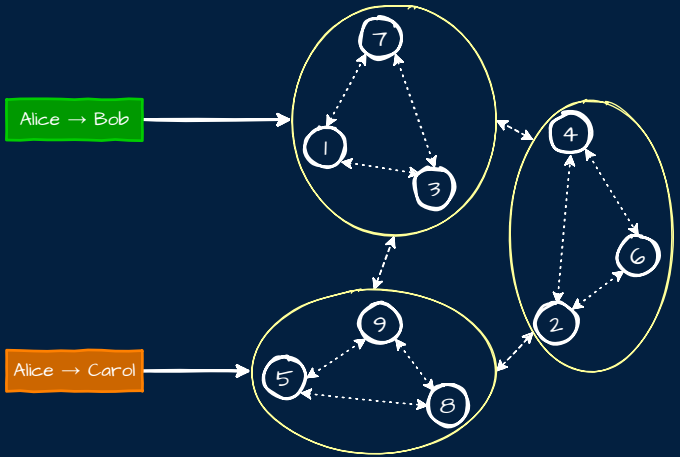
Sharding introduces a security-performance trade-off.



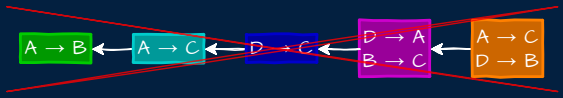
Cross-Shard Validation

Sharding challenges

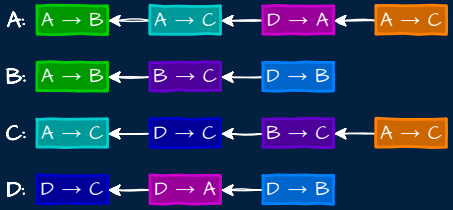
Shards need to communicate when validating transactions that depend on each other.



ScaleGraph

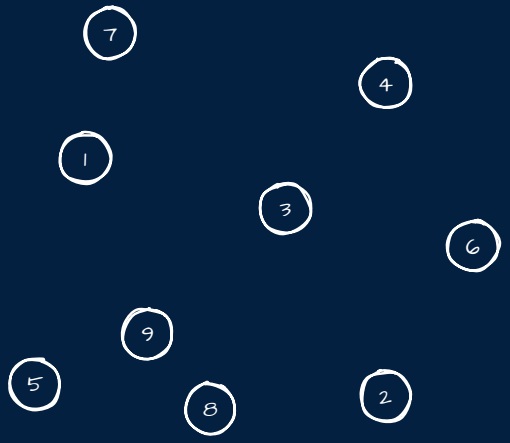


- Full sharding.
- One blockchain per account.
- One transaction per block.
- Note: each transaction appears twice.



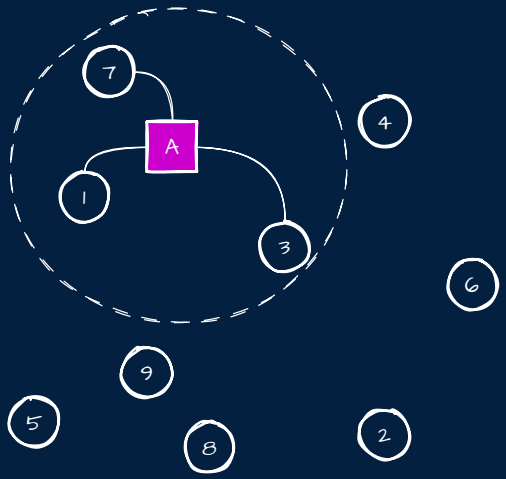
ScaleGraph

- Nodes and accounts have an ID/address.
- Logical distance between nodes is based on their ID.

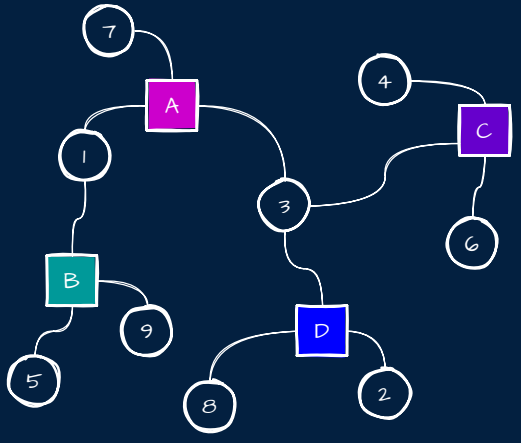


ScaleGraph

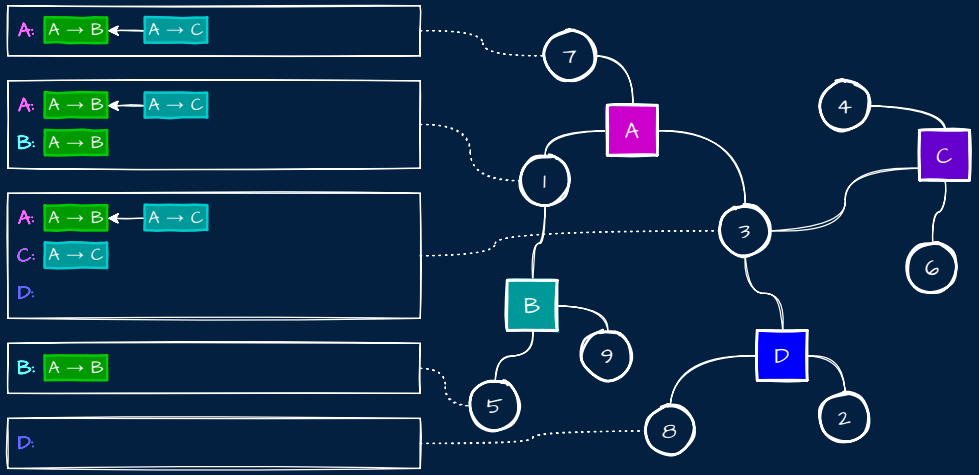
- Accounts have an ID in the same space.
- The v nodes closest to an account form a shard. (E.g. $v = 3$)
- They validate and store transactions related to the account.



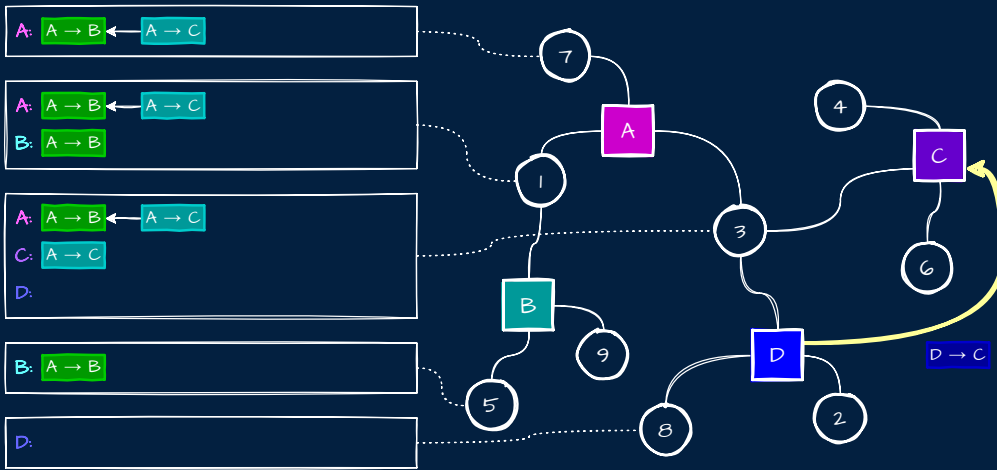
ScaleGraph



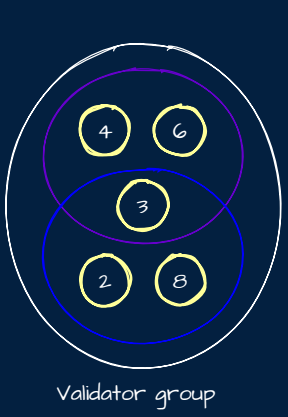
ScaleGraph

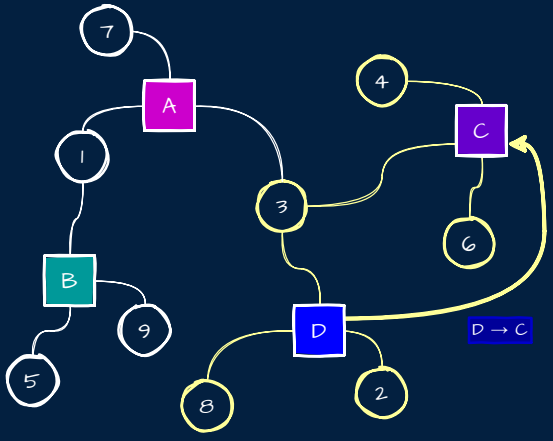


ScaleGraph

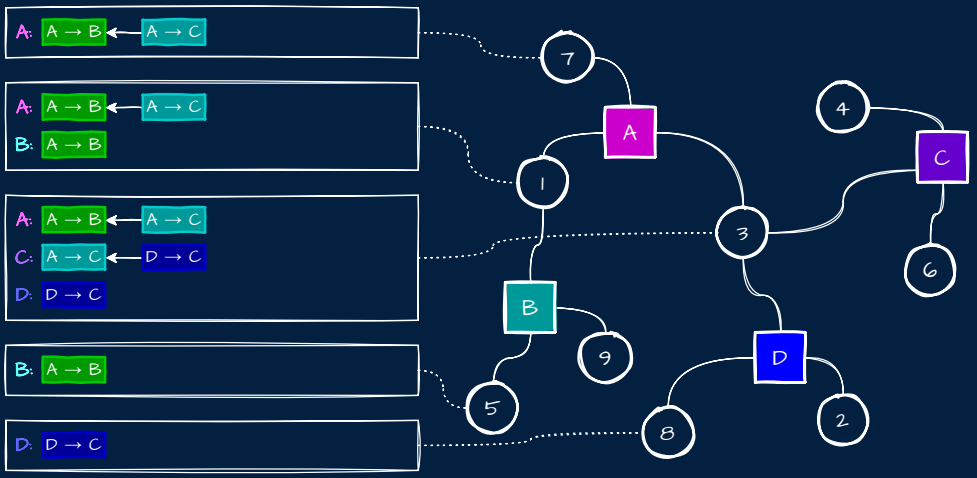


ScaleGraph

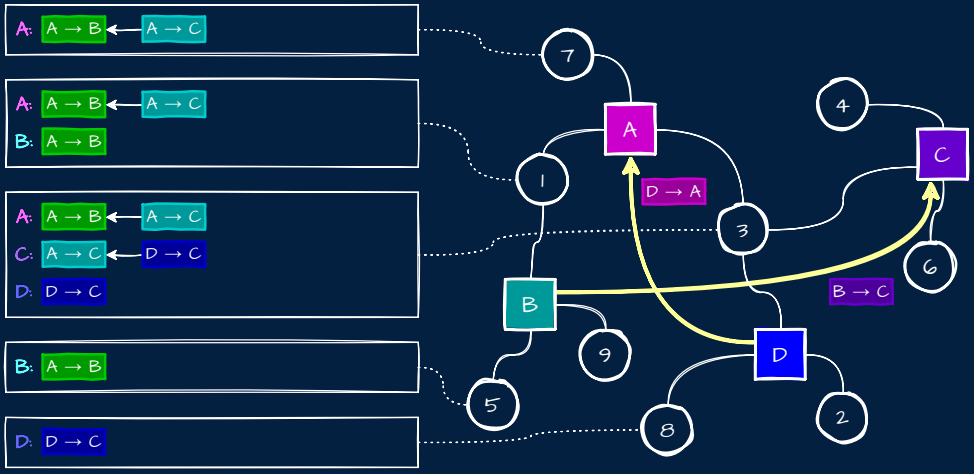




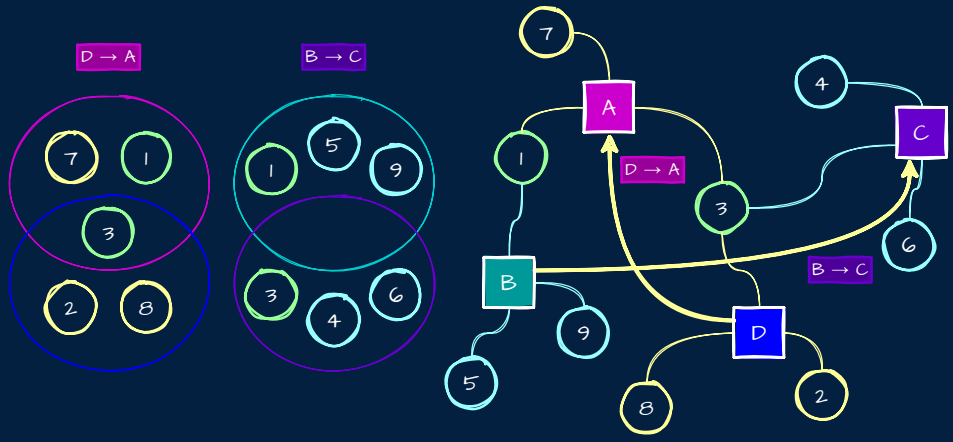
ScaleGraph



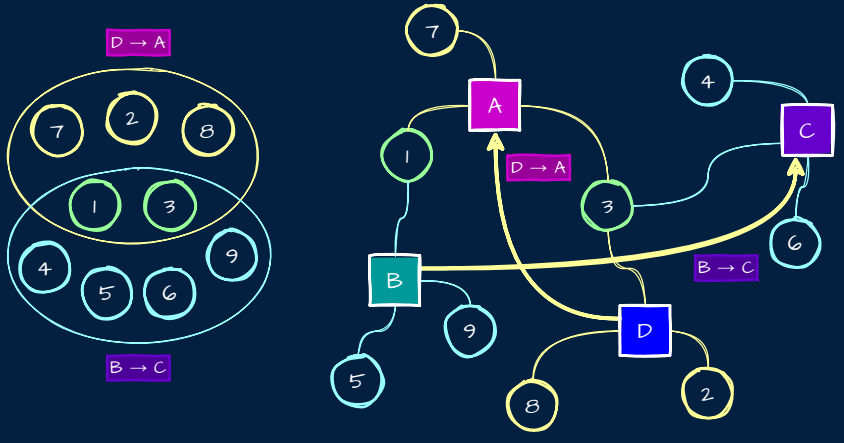
ScaleGraph



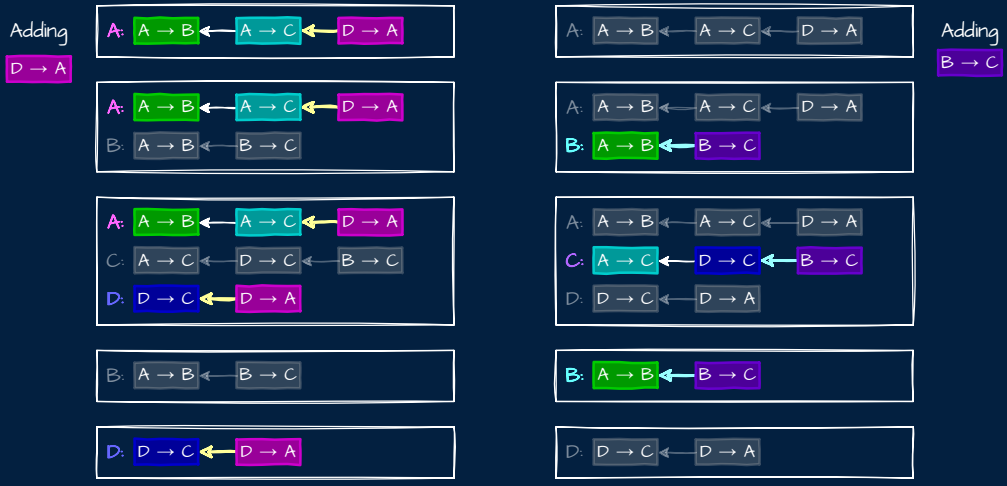
ScaleGraph



ScaleGraph



ScaleGraph


Adding
D → A
A: A → B A → C D → A
A: A → B A → C D → A
B: A → B B → C
A: A → B A → C D → A
C: A → C D → C B → C
D: D → C D → A
B: A → B B → C
D: D → C D → A
A: A → B A → C D → A
Adding
B → C
A: A → B A → C D → A
B: A → B B → C
A: A → B A → C D → A
C: A → C D → C B → C
D: D → C D → A
B: A → B B → C
D: D → C D → A

Key Properties

ScaleGraph

- Any number of disjoint transactions can be validated in parallel.
- Each node can validate transactions for different accounts in parallel.
- (Shards can overlap!)


LULEÅ
UNIVERSITY
OF TECHNOLOGY